



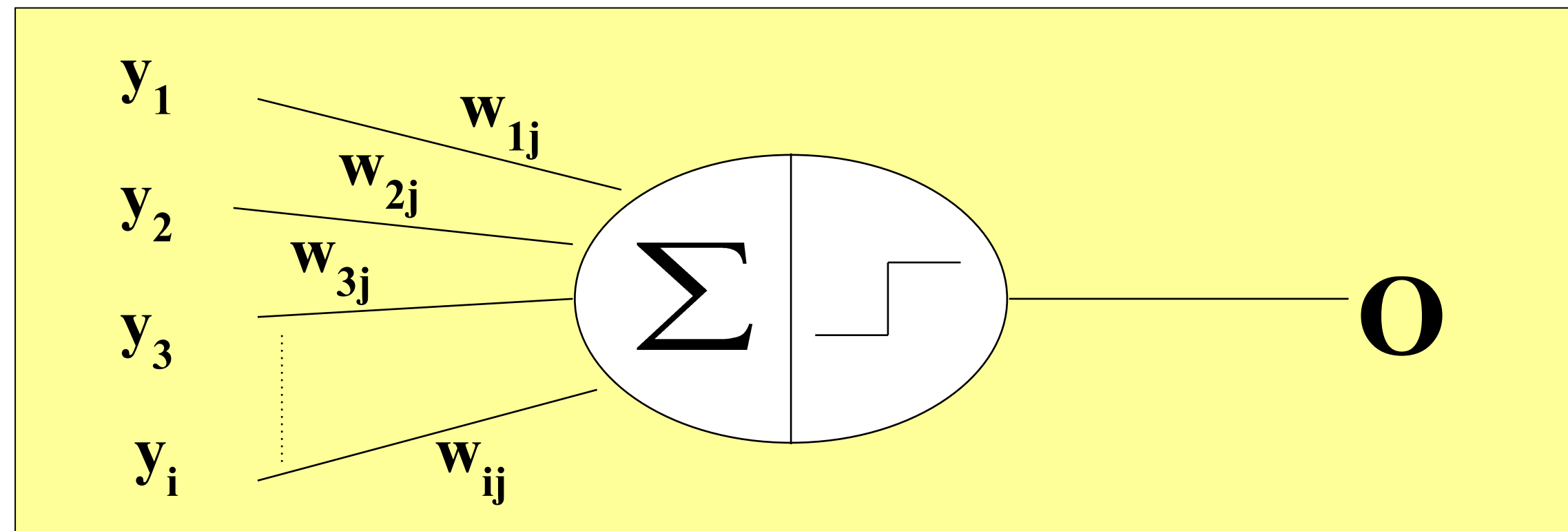
**RV College of
Engineering®**

Go, change the world

Rashtreeya Sikshana Samithi Trust
RV College of Engineering®, Bengaluru.
(An Autonomous Institution Affiliated to VTU, Belagavi)

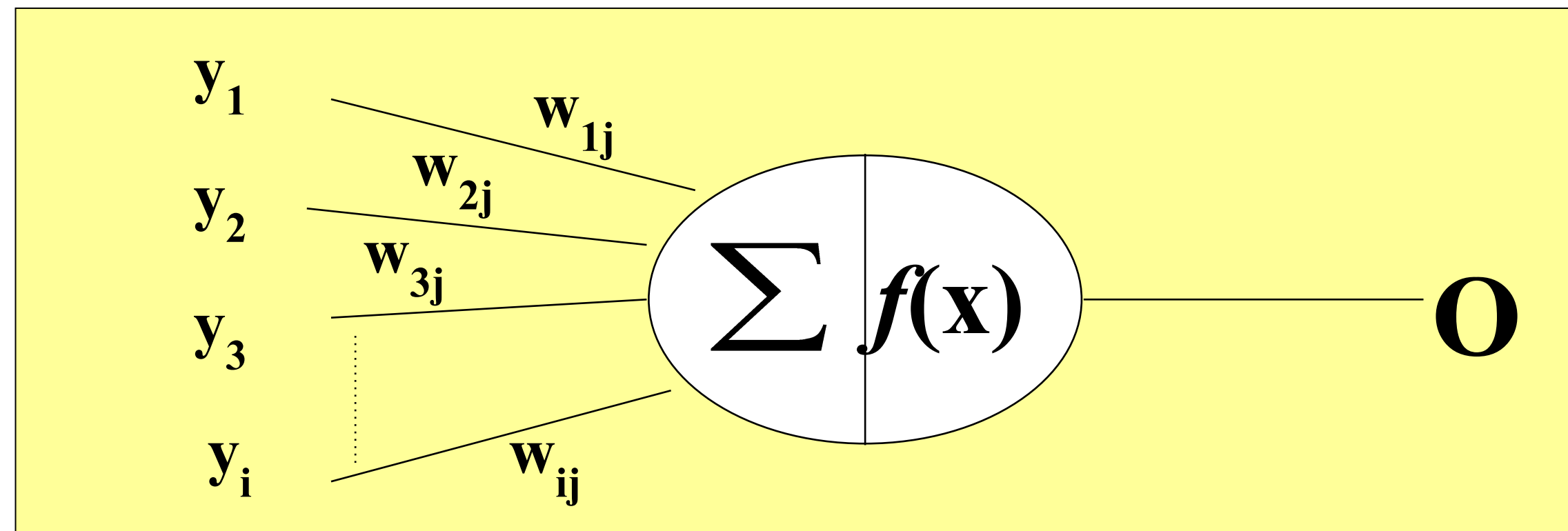
DEEP LEARNING

A Simple Model of a Neuron (Perceptron)



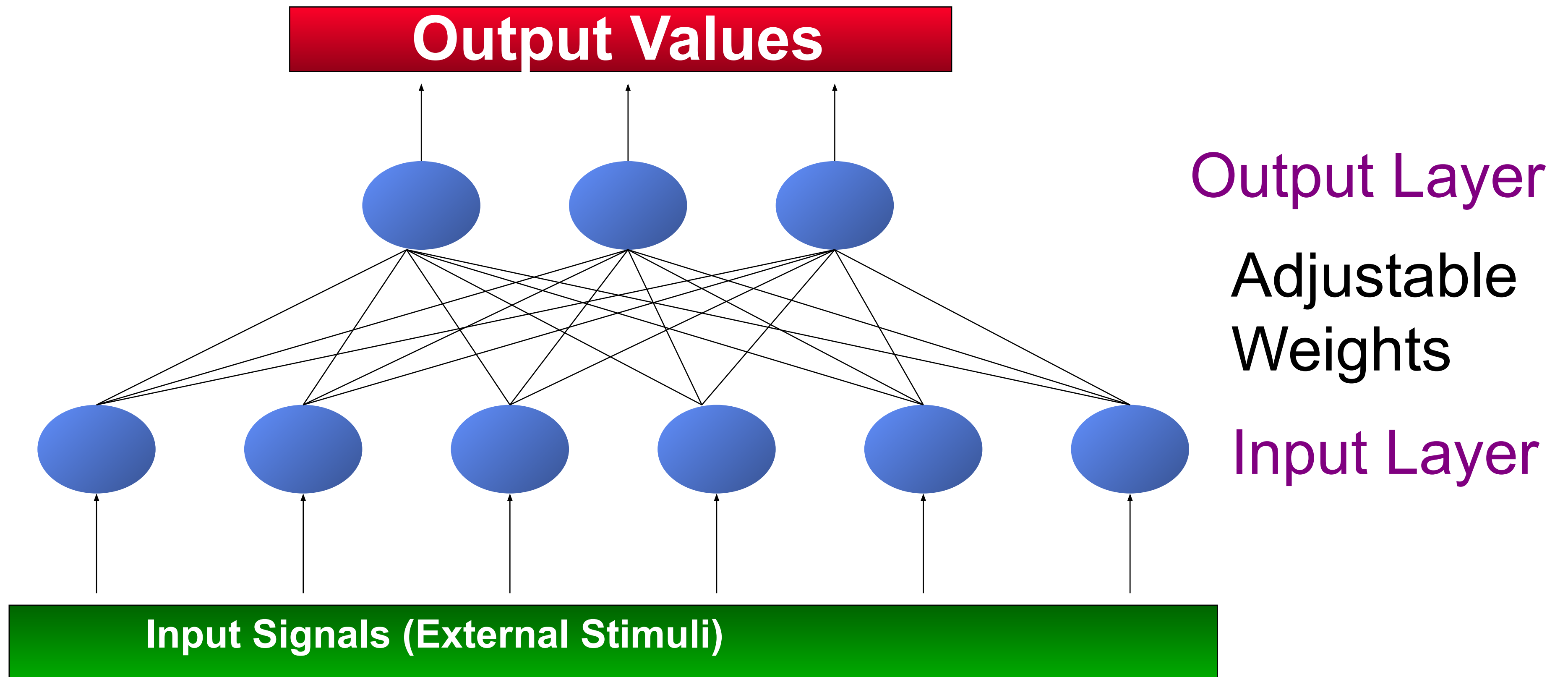
- Each neuron has a threshold value
- Each neuron has weighted inputs from other neurons
- The input signals form a weighted sum
- If the activation level exceeds the threshold, the neuron “fires”

An Artificial Neuron



- Each hidden or output neuron has weighted input connections from each of the units in the preceding layer.
- The unit performs a weighted sum of its inputs, and subtracts its threshold value, to give its activation level.
- Activation level is passed through a sigmoid activation function to determine output.

Multilayer Perceptron (MLP)

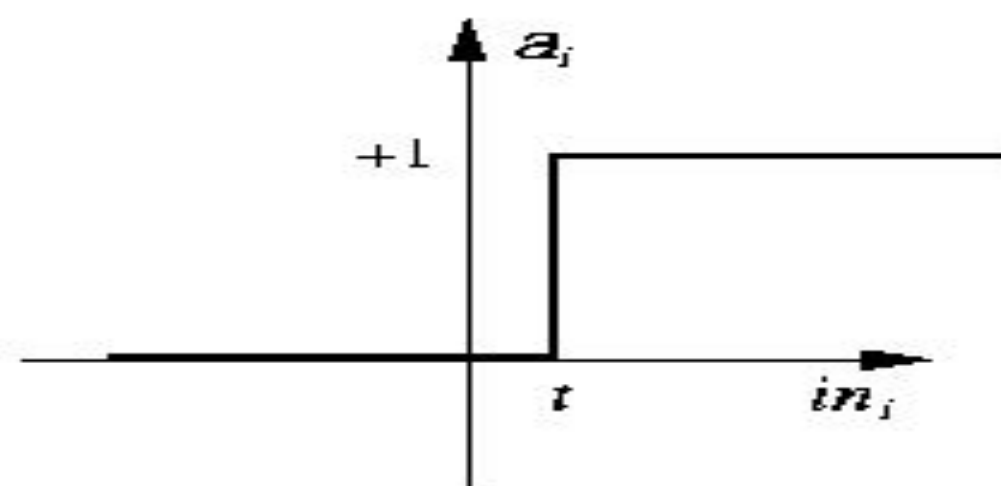


Types of Layers

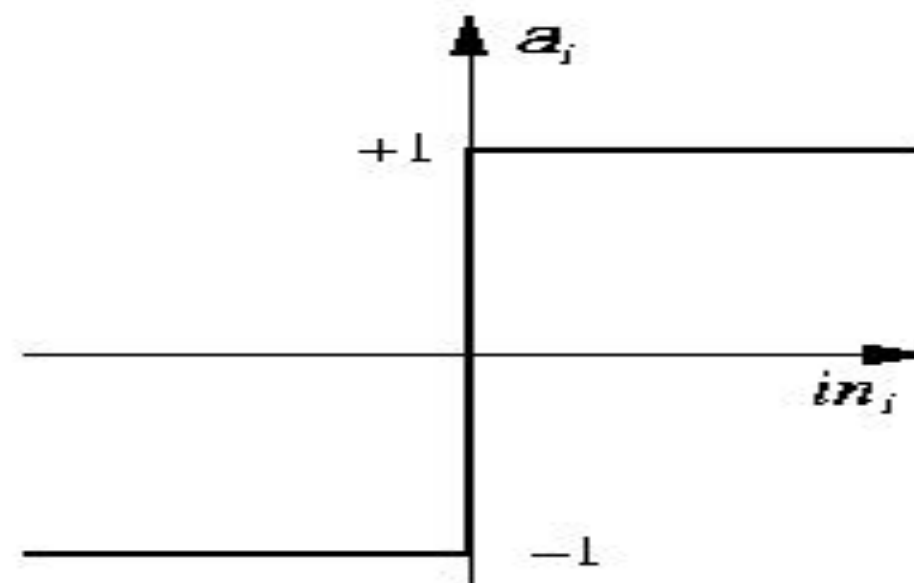
- The input layer.
 - Introduces input values into the network.
 - No activation function or other processing.
- The hidden layer(s).
 - Perform classification of features
 - Two hidden layers are sufficient to solve any problem
 - Features imply more layers may be better
- The output layer.
 - Functionally just like the hidden layers
 - Outputs are passed on to the world outside the neural network.

- Hidden units are nodes that are situated **between the input nodes and the output nodes.**
- Hidden units allow a network **to learn non-linear functions.**
- Hidden units allow the network to represent **combinations of the input features.**

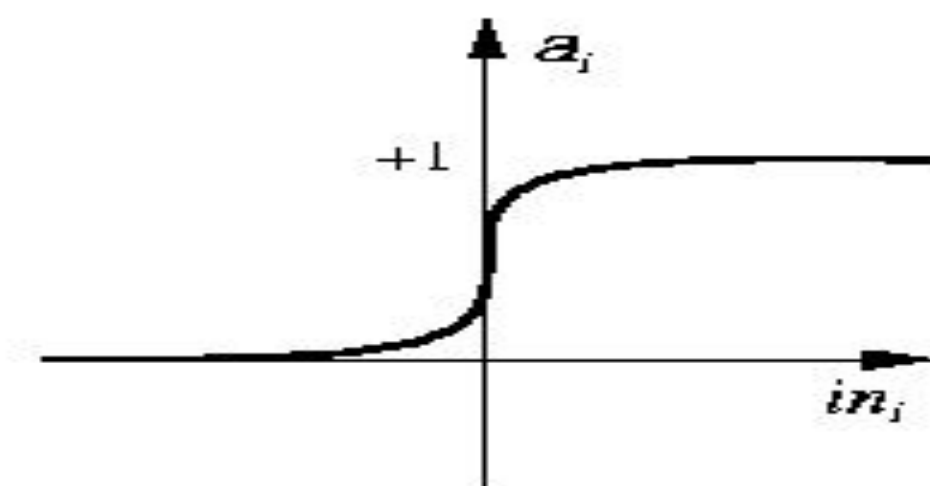
- Transforms neuron's input into output.
- Features of activation functions:
 - A squashing effect is required
 - Prevents accelerating growth of activation levels through the network.



(a) Step function



(b) Sign function



(c) Sigmoid function

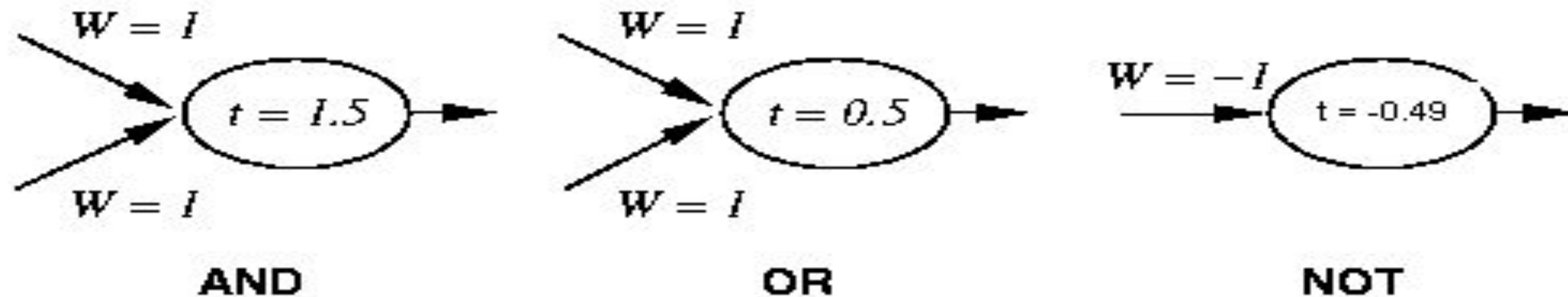
Standard activation functions

- The hard-limiting threshold function
 - Corresponds to the biological paradigm
 - either fires or not
- Sigmoid functions ('S'-shaped curves)
 - The logistic function
 - The hyperbolic tangent (symmetrical)
 - Both functions have a simple differential
 - Only the shape is important



$$\varphi(x) = \frac{1}{1 + e^{-ax}}$$

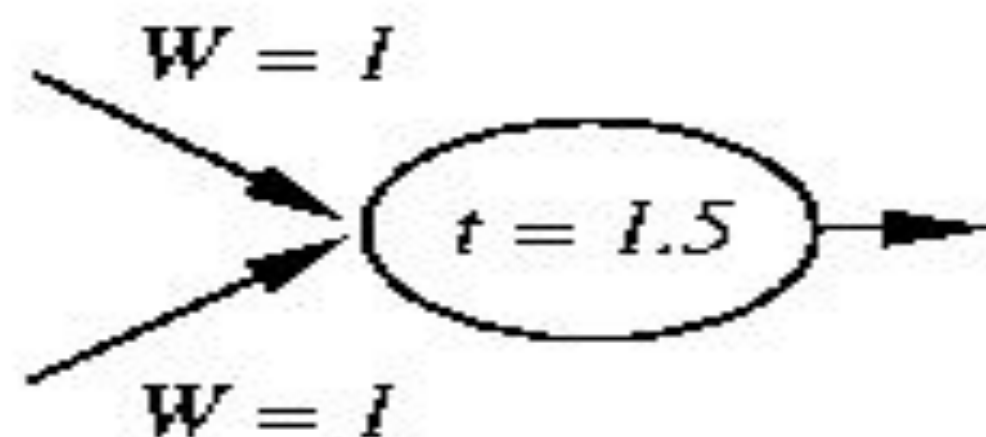
Perceptron Training



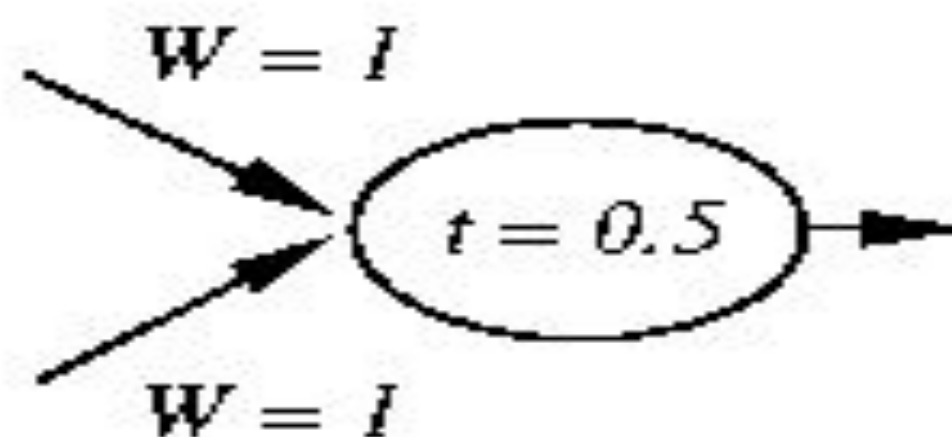
$$\text{Output} = \begin{cases} 1 & \text{if } \sum_{i=0} w_i x_i > t \\ 0 & \text{otherwise} \end{cases}$$

- Linear threshold is used.
- W - weight value
- t - threshold value

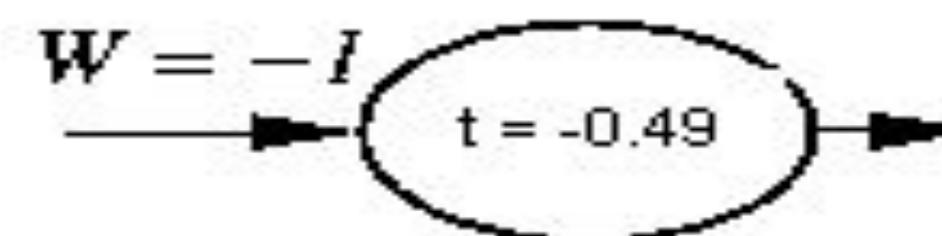
Simple network



AND

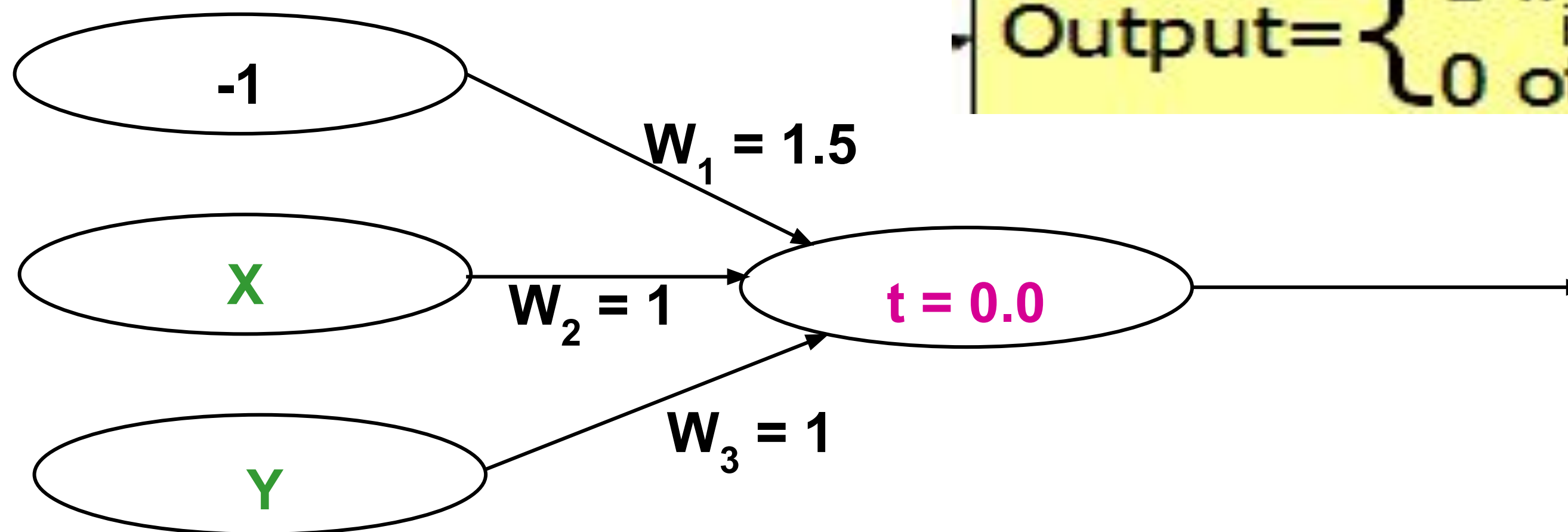


OR



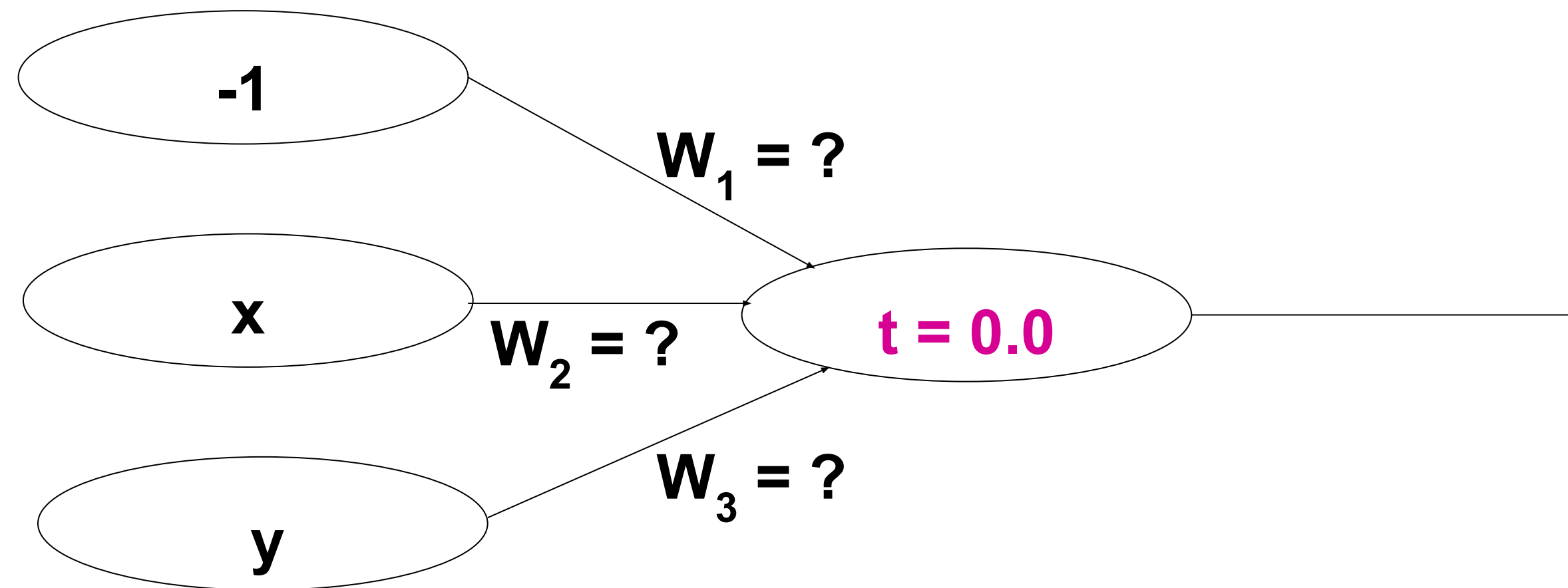
NOT

AND with a Biased input



$$\text{Output} = \begin{cases} 1 & \text{if } \sum_{i=0} w_i x_i > t \\ 0 & \text{otherwise} \end{cases}$$

Training Perceptrons

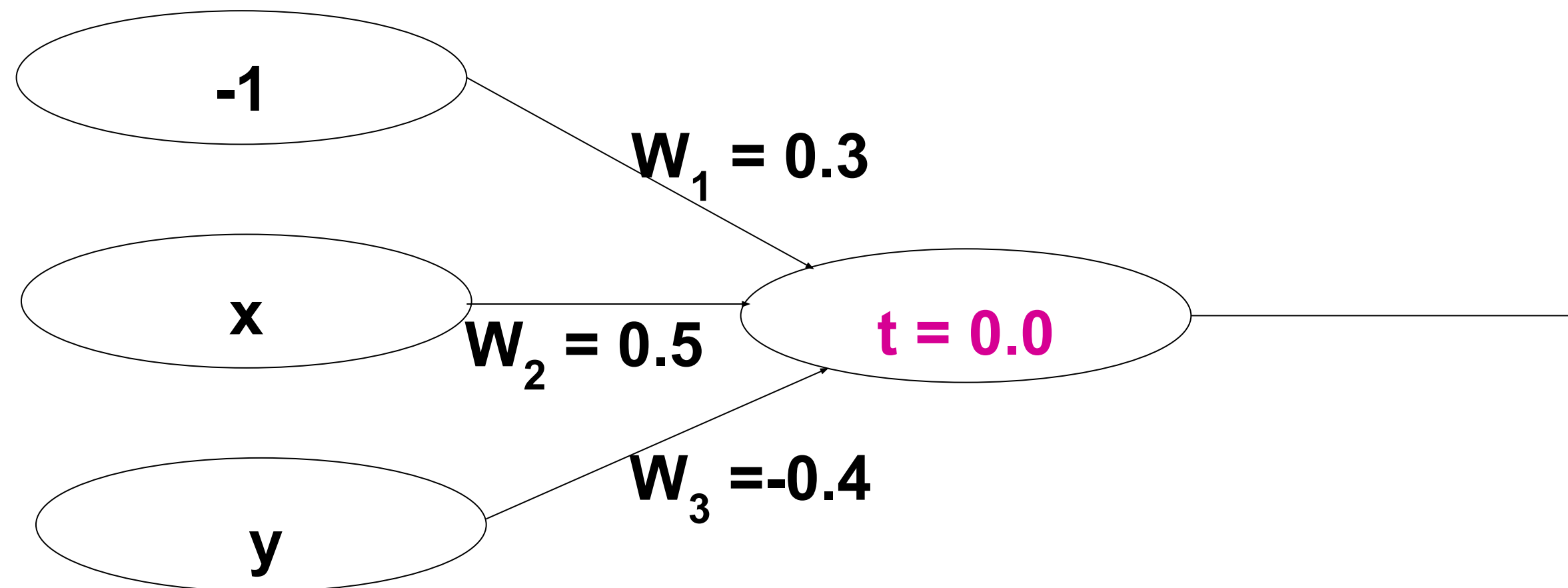


For AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

- What are the weight values?
- Initialize with random weight values

Training Perceptrons



For AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

I_1	I_2	I_3	Summation	Output
-1	0	0	$(-1*0.3) + (0*0.5) + (0*-0.4) = -0.3$	0
-1	0	1	$(-1*0.3) + (0*0.5) + (1*-0.4) = -0.7$	0
-1	1	0	$(-1*0.3) + (1*0.5) + (0*-0.4) = 0.2$	1
-1	1	1	$(-1*0.3) + (1*0.5) + (1*-0.4) = -0.2$	0

- Perceptron can be used to represent the following Boolean functions
 - AND
 - OR
 - Any m-of-n function
 - NOT
 - NAND (NOT AND)
 - NOR (NOT OR)
- Every Boolean function can be represented by a network of interconnected units based on these primitives
 - Two levels (i.e., **one hidden layer**) is enough!!!

Boolean XOR

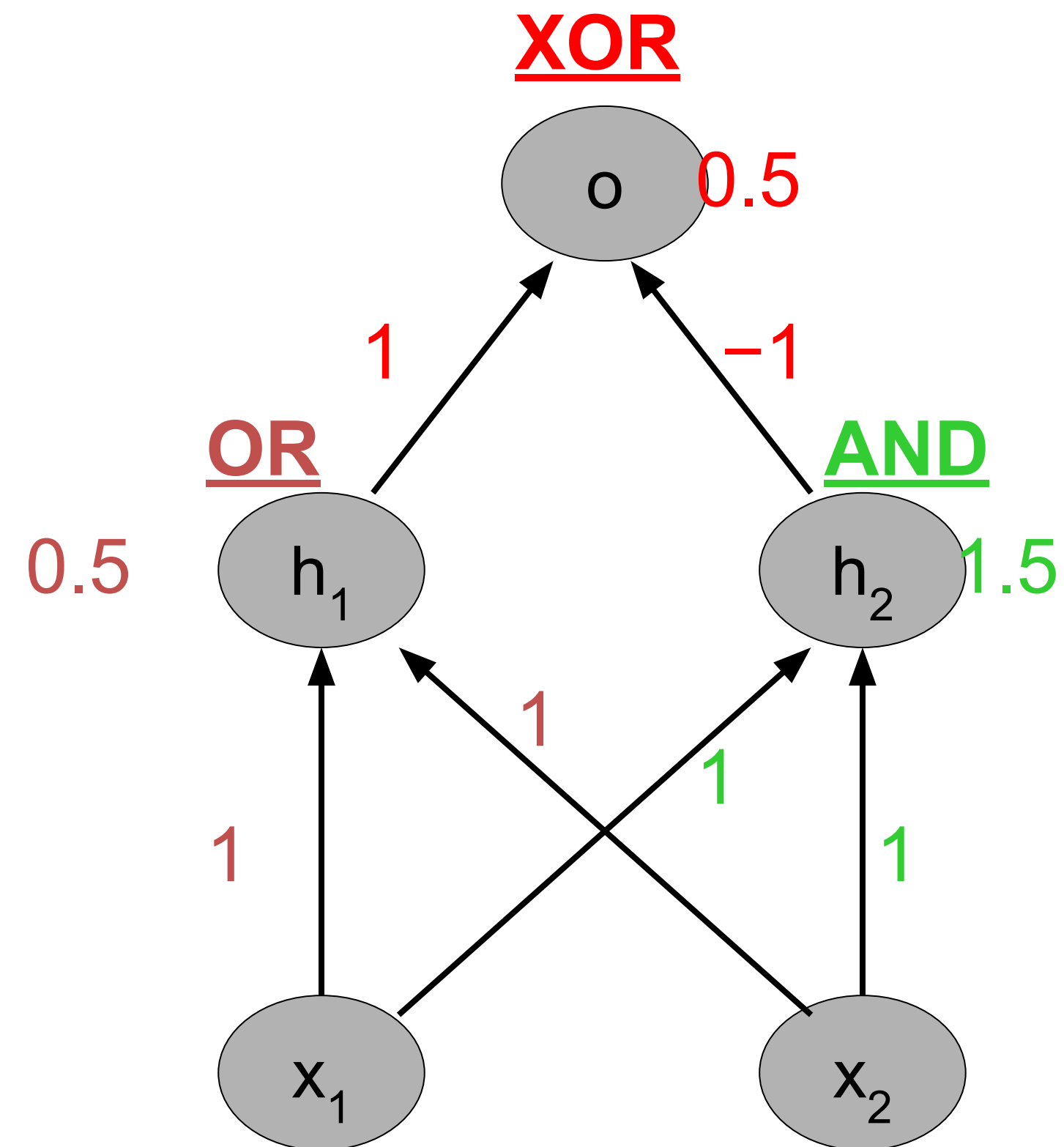
$$X1 \oplus X2 \Leftrightarrow (X1 \vee X2) \wedge \neg(X1 \wedge X2)$$

Not Linear separable □

Cannot be represented by a
single-layer perceptron

Let's consider a **single hidden layer**
network, using as **building blocks**
threshold units.

$$w_1 x_1 + w_2 x_2 - w_0 > 0$$



Deep feedforward networks feedforward, also often called neural networks, or multilayer perceptrons (MLPs), are the deep learning models.

The goal of a feedforward network is to approximate some function f^ . For example, for a classifier,*

$y = f^(x)$ maps an input x to a category y .*

A feedforward network defines a mapping $y = f(x; \theta)$ and learns the value of the parameters θ that result in the best function approximation.

These models are called feedforward because information flows through the function being evaluated from x , through the intermediate computations used to define f , and finally to the output y .

There are no feedback connections in which outputs of the model are fed back into itself.

When feedforward neural networks are extended to include feedback connections, they are called recurrent neural networks,

The model is associated with a directed acyclic graph describing how the functions are composed together.

For example, we might have three functions $f^{(1)}$, $f^{(2)}$, and $f^{(3)}$ connected in a chain, to form $f(x) = f^{(3)}(f^{(2)}(f^{(1)}(x)))$. These chain structures are the most commonly used structures of neural networks. In this case, $f^{(1)}$ is called the *first layer* of the network, $f^{(2)}$ is called the *second layer*, and so on. The overall length

of the chain gives the of the model. It is from this *depth terminology that the* name “deep learning” arises. The final layer of a feedforward network is called the *output layer*.

During neural network training, we drive $f(x)$ to match $f^*(x)$.

The training data provides us with noisy, approximate examples of $f^(x)$ evaluated at different training points. Each example x is accompanied by a label $y \approx f^*(x)$.*

The training examples specify directly what the output layer must do at each point x ; *it must produce a value that is close to y .*

The behavior of the other layers is not directly specified by the training data. The learning algorithm must decide how to use those layers to produce the desired output, but the training data does not say what each individual layer should do.

Instead, the learning algorithm must decide how to use these layers to best implement an approximation of f^* . *Because* the training data does not show the desired output for each of these layers, these layers are called *hidden layers*

Each hidden layer of the network is typically vector-valued. The dimensionality of these hidden layers determines the *width of the model*

we can also think of the layer as consisting of many *units that act in parallel*, each representing a vector-to-scalar function.

feedforward networks begin with linear models and consider how to overcome their limitations.

Linear models also have the obvious defect that the model capacity is limited to linear functions

To extend linear models to represent nonlinear functions of x , we can apply the linear model not to x itself but to a transformed input $\phi(x)$, where ϕ is a nonlinear transformation. We can think of ϕ as providing a set of features describing x , or as providing a new representation for x .

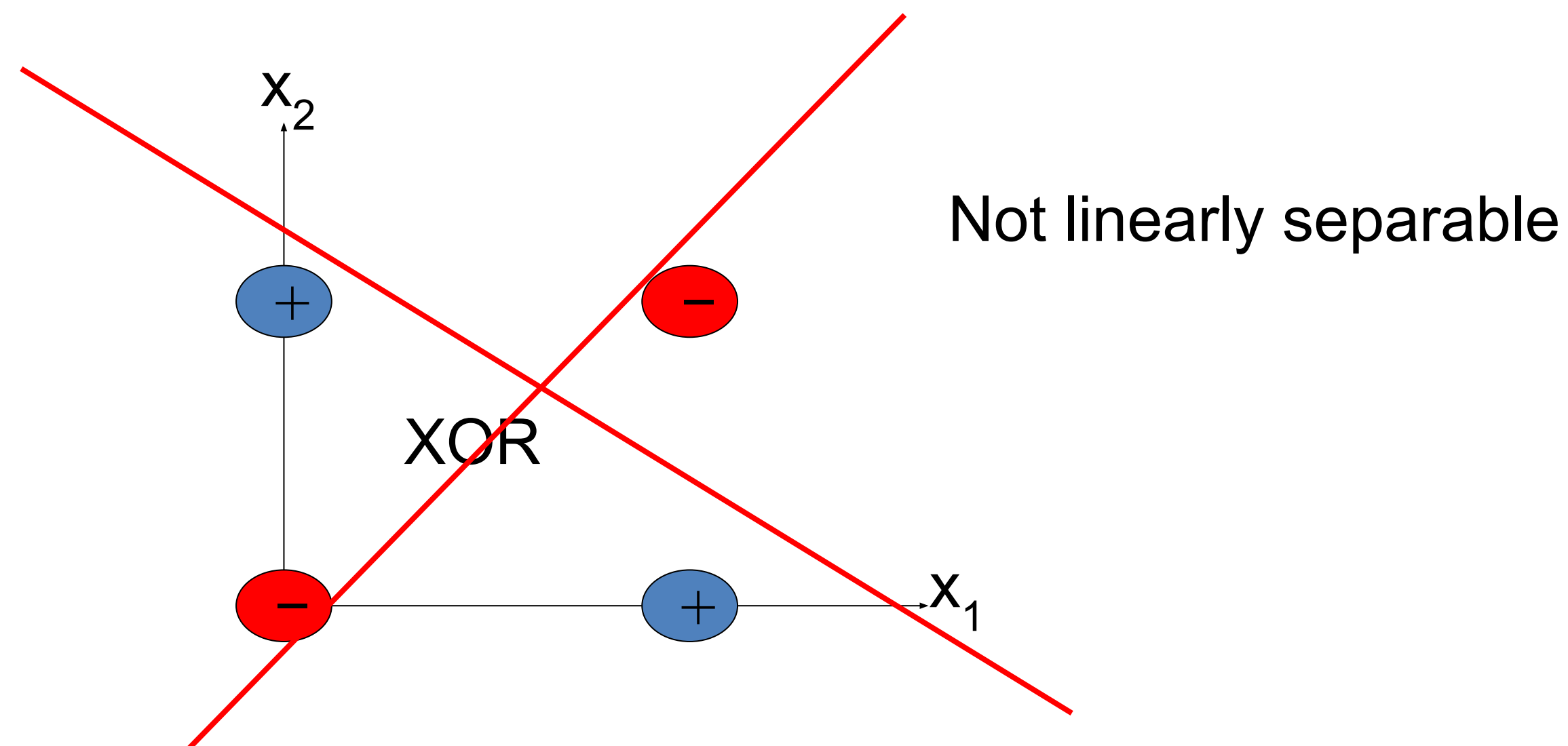
The question is then how to choose the mapping ϕ

1. One option is to use a very generic ϕ , such as the infinite-dimensional ϕ that is implicitly used by kernel machines based on the RBF kernel. If $\phi(\mathbf{x})$ is of high enough dimension, we can always have enough capacity to fit the training set, but generalization to the test set often remains poor. Very
2. Another option is to manually engineer ϕ . Until the advent of deep learning, this was the dominant approach. It requires decades of human effort for each separate task, with practitioners specializing in different domains, such as speech recognition or computer vision, and with little transfer between domains.

The strategy of deep learning is to learn Φ . In this approach, we have a model $y = f(x; \theta, w) = \varphi(x; \theta)^T w$. We now have parameters θ that we use to learn φ from a broad class of functions, and parameters w that map from $\varphi(x)$ to the desired output. This is an example of a deep feedforward network, with φ defining a hidden layer

Perceptron:

Linear Separable Functions

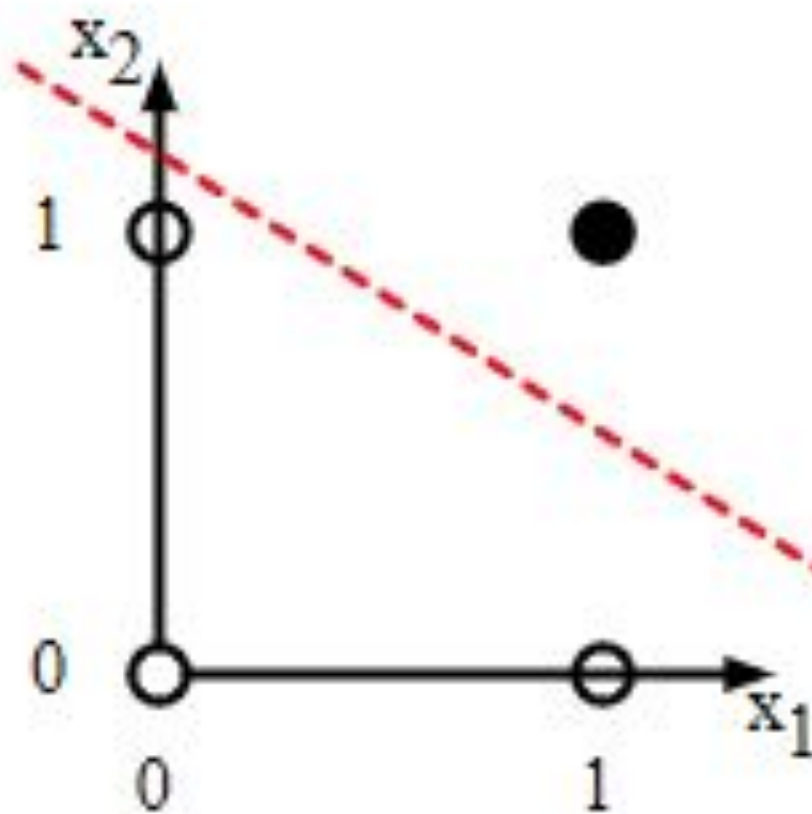


Minsky & Papert (1969)

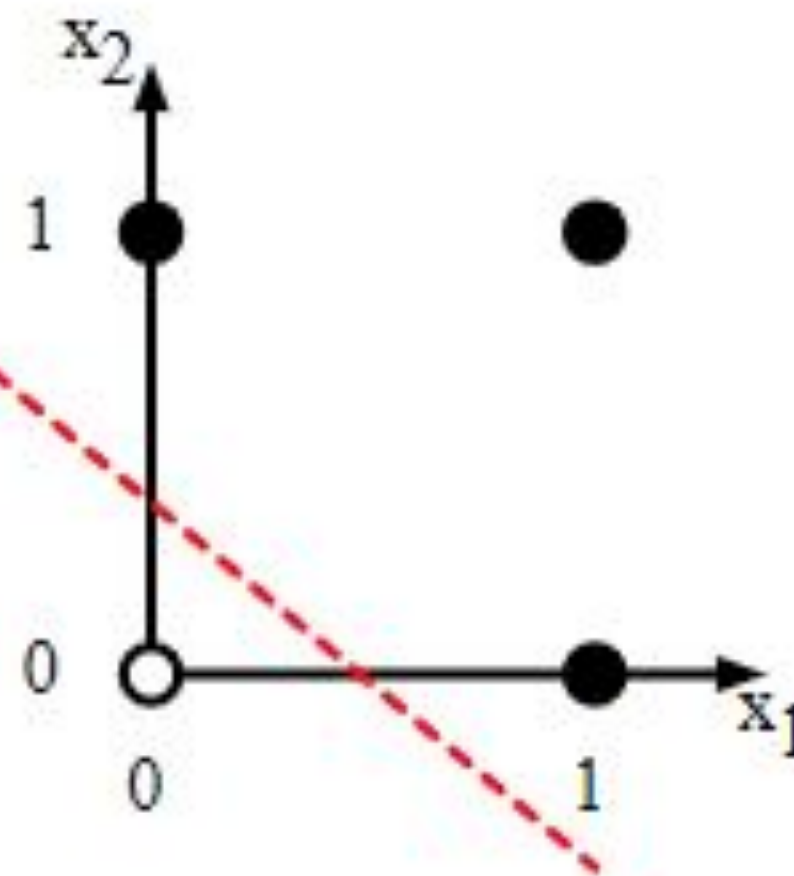
Perceptrons can only represent linearly separable functions.

The XOR problem

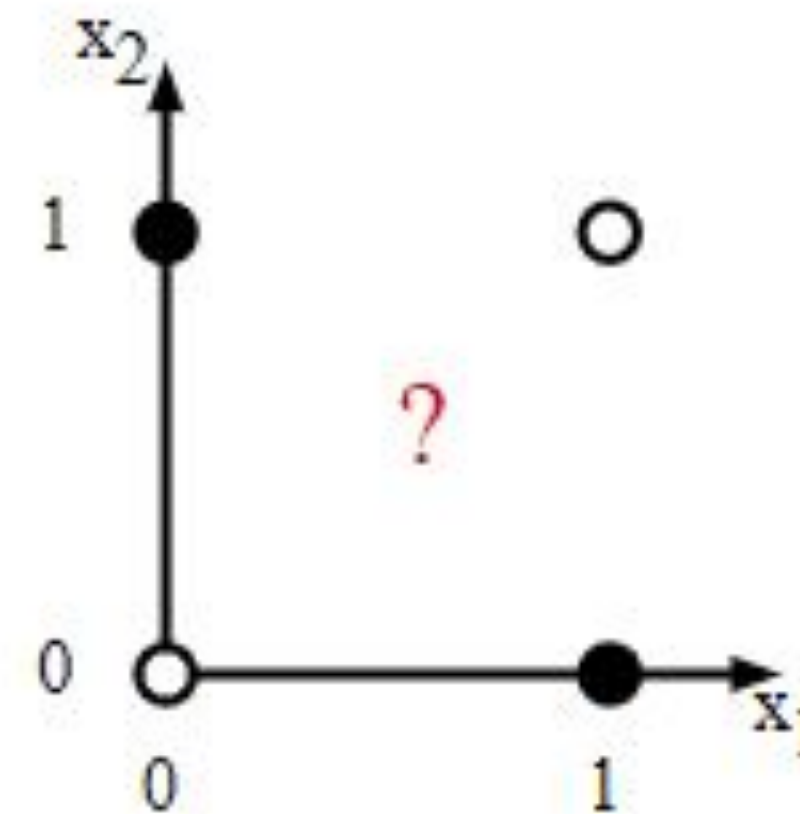
While AND / OR are linearly separable, the XOR problem is not



a) x_1 AND x_2



b) x_1 OR x_2



c) x_1 XOR x_2

The XOR function provides the target function

We want our network to perform correctly on the four points $\mathbb{X} = \{[0,0]^\top, [0,1]^\top, [1,0]^\top, \text{ and } [1,1]^\top\}$. We will train the network on all four of these points. The only challenge is to fit the training set.

Evaluated on our whole training set, the MSE loss function is

$$J(\boldsymbol{\theta}) = \frac{1}{4} \sum_{\mathbf{x} \in \mathbb{X}} (f^*(\mathbf{x}) - f(\mathbf{x}; \boldsymbol{\theta}))^2. \quad (6.1)$$

$n=N$

Now we must choose the form of our model, $f(\mathbf{x}; \boldsymbol{\theta})$. Suppose that we choose a linear model, with $\boldsymbol{\theta}$ consisting of $\mathbf{w}$ and b . Our model is defined to be

$$f(\mathbf{x}; \mathbf{w}, b) = \mathbf{x}^\top \mathbf{w} + b. \quad (6.2)$$

We can minimize $J(\boldsymbol{\theta})$ in closed form with respect to $\mathbf{w}$ and b using the normal equations.

After solving the normal equations, we obtain $\mathbf{w} = \mathbf{0}$ and $b = \frac{1}{2}$. The linear model simply outputs 0.5 everywhere. Why does this happen? Figure 6.1 shows how a linear model is not able to represent the XOR function. One way to solve this problem is to use a model that learns a different feature space in which a linear model is able to represent the solution.

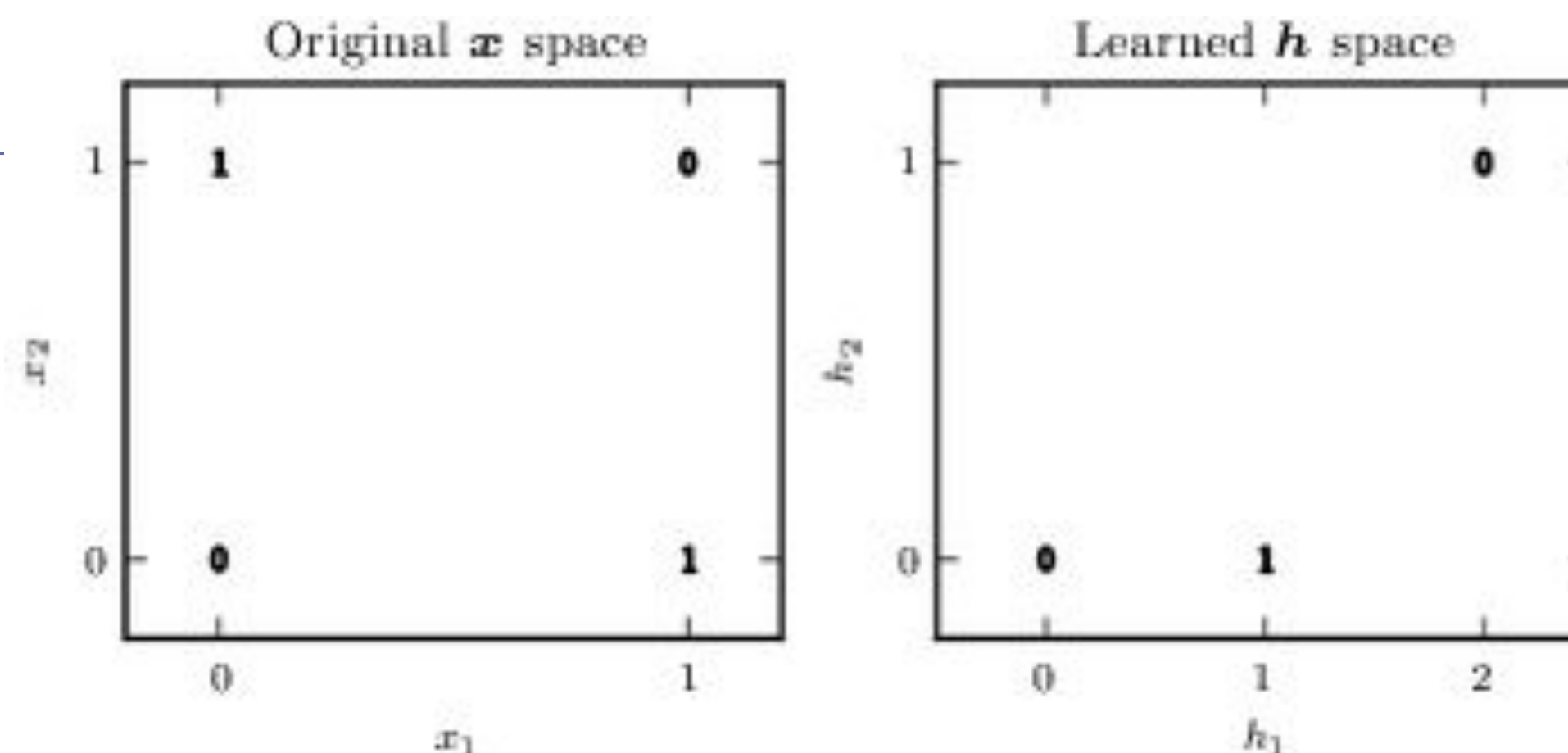


Figure 6.1: Solving the XOR problem by learning a representation. The bold numbers printed on the plot indicate the value that the learned function must output at each point. *(Left)* A linear model applied directly to the original input cannot implement the XOR function. When $x_1 = 0$, the model's output must increase as x_2 increases. When $x_1 = 1$, the model's output must decrease as x_2 increases. A linear model must apply a fixed coefficient w_2 to x_2 . The linear model therefore cannot use the value of x_1 to change the coefficient on x_2 and cannot solve this problem. *(Right)* In the transformed space represented by the features extracted by a neural network, a linear model can now solve the problem. In our example solution, the two points that must have output 1 have been collapsed into a single point in feature space. In other words, the nonlinear features have mapped both $\mathbf{x} = [1, 0]^\top$ and $\mathbf{x} = [0, 1]^\top$ to a single point in feature space, $\mathbf{h} = [1, 0]^\top$. The linear model can now describe the function as increasing in h_1 and decreasing in h_2 . In this example, the motivation for learning the feature space is only to make the model capacity greater so that it can fit the training set. In more realistic applications, learned representations can also help the model to generalize.

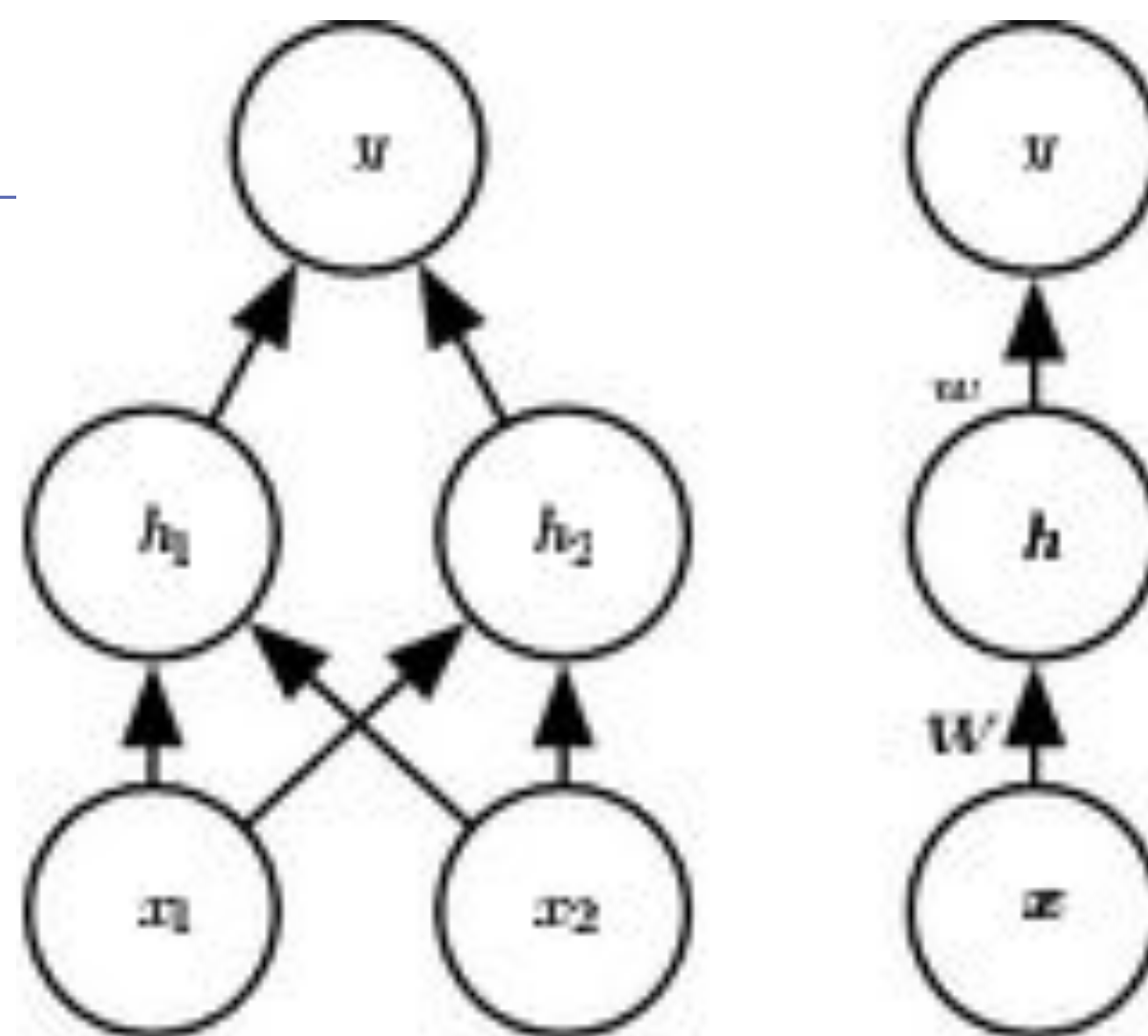


Figure 6.2: An example of a feedforward network, drawn in two different styles. Specifically, this is the feedforward network we use to solve the XOR example. It has a single hidden layer containing two units. *(Left)* In this style, we draw every unit as a node in the graph. This style is explicit and unambiguous, but for networks larger than this example, it can consume too much space. *(Right)* In this style, we draw a node in the graph for each entire vector representing a layer's activations. This style is much more compact. Sometimes we annotate the edges in this graph with the name of the parameters that describe the relationship between two layers. Here, we indicate that a matrix W describes the mapping from x to h , and a vector w describes the mapping from h to y . We typically omit the intercept parameters associated with each layer when labeling this kind of drawing.

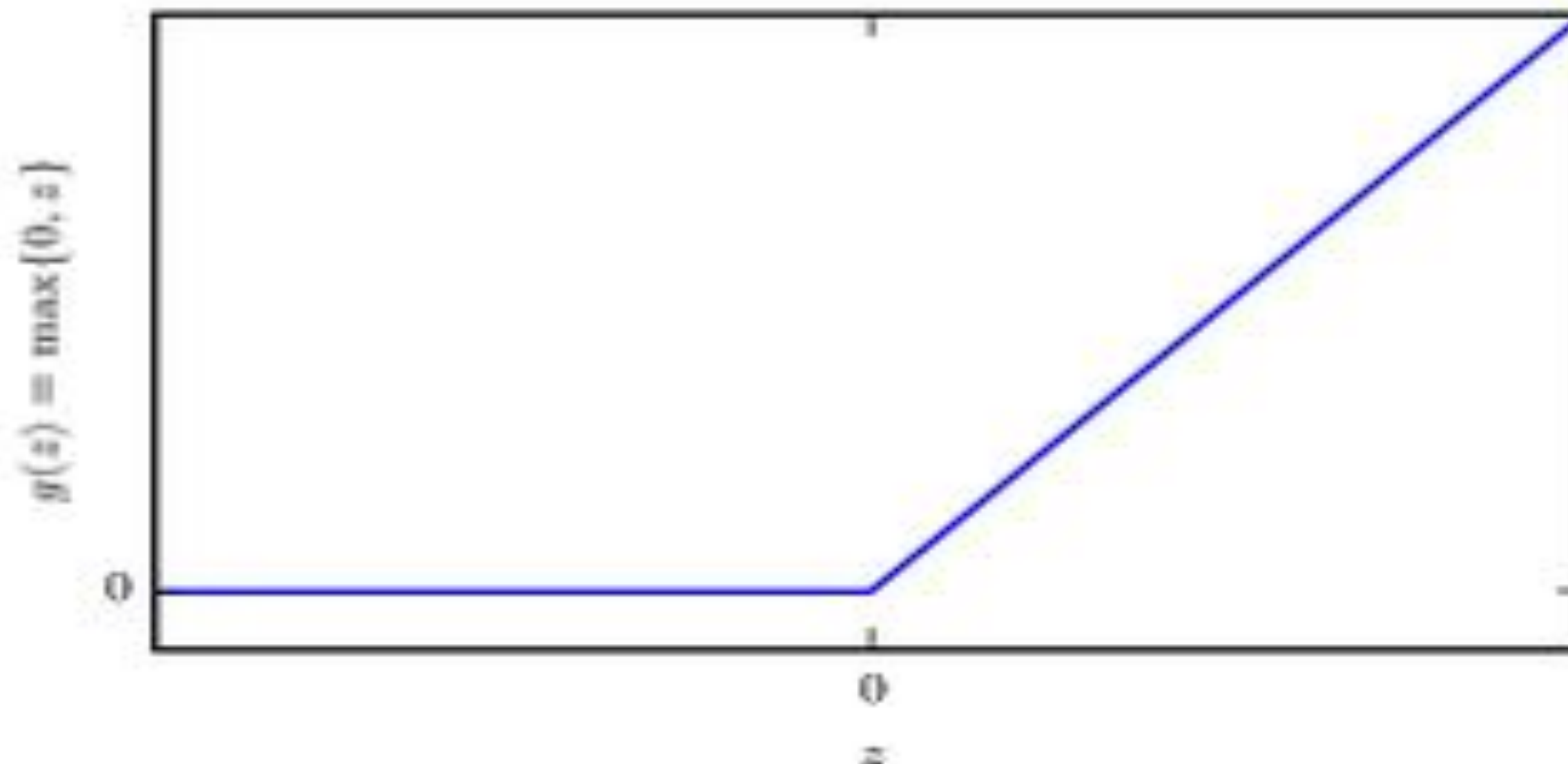


Figure 6.3: The rectified linear activation function. This activation function is the default activation function recommended for use with most feedforward neural networks. Applying this function to the output of a linear transformation yields a nonlinear transformation. The function remains very close to linear, however, in the sense that it is a piecewise linear function with two linear pieces. Because rectified linear units are nearly linear, they preserve many of the properties that make linear models easy to optimize with gradient-based methods. They also preserve many of the properties that make linear models generalize well. A common principle throughout computer science is that we can build complicated systems from minimal components. Much as a Turing machine's memory needs only to be able to store 0 or 1 states, we can build a universal function approximator from rectified linear functions.

In modern neural networks, the default recommendation is to use the rectified linear unit , or ReLU

We can now specify our complete network as

$$f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = \mathbf{w}^\top \max\{0, \mathbf{W}^\top \mathbf{x} + \mathbf{c}\} + b. \quad (6.3)$$

We can then specify a solution to the XOR problem. Let

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad (6.4)$$

$$\mathbf{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad (6.5)$$

$$\mathbf{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad (6.6)$$

and $b = 0$.

We can now walk through how the model processes a batch of inputs. Let $\mathbf{X}$ be the design matrix containing all four points in the binary input space, with one example per row:

$$\mathbf{X} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}. \quad (6.7)$$

The first step in the neural network is to multiply the input matrix by the first layer's weight matrix:

$$\mathbf{XW} = \begin{bmatrix} 0 & 0 \\ 1 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix}. \quad (6.8)$$

Next, we add the bias vector $\mathbf{c}$, to obtain

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}. \quad (6.9)$$

In this space, all the examples lie along a line with slope 1. As we move along this line, the output needs to begin at 0, then rise to 1, then drop back down to 0. A linear model cannot implement such a function. To finish computing the value of h for each example, we apply the rectified linear transformation:

$$\begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 2 & 1 \end{bmatrix}. \quad (6.10)$$

This transformation has changed the relationship between the examples. They no longer lie on a single line. As shown in figure 6.1, they now lie in a space where a linear model can solve the problem.

We finish with multiplying by the weight vector w :

$$\begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}. \quad (6.11)$$

The neural network has obtained the correct answer for every example in the batch.

In this example, we simply specified the solution, then showed that it obtained zero error. In a real situation, there might be billions of model parameters and billions of training examples, so one cannot simply guess the solution as we did here. Instead, a gradient-based optimization algorithm can find parameters that produce very little error. The solution we described to the XOR problem is at a global minimum of the loss function, so gradient descent could converge to this point. There are other equivalent solutions to the XOR problem that gradient descent could also find. The convergence point of gradient descent depends on the initial values of the parameters. In practice, gradient descent would usually not find clean, easily understood, integer-valued solutions like the one we presented here.

Deep **learning** neural networks are trained using the stochastic **gradient descent** optimization algorithm. The **learning** rate is a hyperparameter that controls how much to change the model in response to the estimated error each time the model weights are updated

Gradient Descent is an optimization algorithm for finding a local minimum of a differentiable function. **Gradient descent** is simply used to find the values of a function's parameters (coefficients) that minimize a cost function as far as possible.

Computing the gradient is slightly more complicated for a neural network, but can still be done efficiently and exactly. Sec. 6.5 will describe how to obtain the gradient using the back-propagation algorithm and modern generalizations of the back-propagation algorithm.

As with other machine learning models, to apply gradient-based learning we must choose **a cost function**, and we must choose how to represent the **output of the model**. We now revisit these design considerations with special emphasis on the neural networks scenario.

For the moment, it suffices to understand that the training algorithm is almost always based on using the gradient to descend the cost function in one way or another.

The specific algorithms are improvements and refinements on the ideas of gradient descent, most often improvements of the stochastic gradient descent algorithm, (introduced in section 5.9).

Computing the gradient is slightly more complicated for a neural network but can still be done efficiently and exactly - using the **back-propagation algorithm and modern generalizations of the back-propagation algorithm**.

6.2.1 Cost Functions: the cost functions for neural networks In most cases, our parametric model defines a distribution $p(y \mid x; \theta)$ and we simply use the principle of maximum likelihood. This means we use the cross-entropy between the training data and the model's predictions as the cost function.

Sometimes, we take a simpler approach, where rather than predicting a complete probability distribution over y , we merely predict some statistic of y conditioned on x .

Most modern neural networks are trained using **maximum likelihood**. This means that the cost function is simply the negative log-likelihood, equivalently described as the cross-entropy between the training data and the model distribution.

An advantage of this approach of deriving the cost function from maximum likelihood is that it removes the burden of designing cost functions for each model. Specifying a model $P(y | x)$ automatically determines a cost function $\log p(y | x)$

Note:

<https://towardsdatascience.com/probability-concepts-explained-maximum-likelihood-estimation-c7b4342fdbb1>

6.3 Hidden Units : The design of hidden units is an extremely active area of research and does not yet have many definitive guiding theoretical principles.

Unless indicated otherwise, most hidden units can be described as accepting a vector of inputs $\mathbf{x}$, computing an affine transformation $\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b}$, and then applying an element-wise nonlinear function $g(\mathbf{z})$. Most hidden units are distinguished from each other only by the choice of the form of the activation function $g(\mathbf{z})$.

Rectified Linear Units and Their Generalizations : Rectified linear units use the activation function $g(z) = \max\{0, z\}$. The only difference between a linear unit and a rectified linear unit is that a rectified linear unit outputs zero across half its domain. This makes the derivatives through a rectified linear unit remain large whenever the unit is active.

The gradients are not only large but **also consistent**. The second derivative of the rectifying operation is 0 almost everywhere, and the derivative of the rectifying operation is 1 everywhere that the unit is active. This means that the gradient direction is far more useful for learning than it would be with activation functions that introduce second-order effects.

Three generalizations of rectified linear units are based on using a nonzero

$$\text{slope } \alpha_i \text{ when } z_i < 0: h_i = g(\mathbf{z}, \boldsymbol{\alpha})_i = \max(0, z_i) + \alpha_i \min(0, z_i).$$

Actual rectification

rectification fixes $\alpha_i = -1$ to obtain $g(z) = |z|$. It is used for object recognition from images (Jarrett *et al.*, 2009), where it makes sense to seek features that are invariant under a polarity reversal of the input illumination. Other generalizations of rectified linear units are more broadly applicable. A **leaky ReLU** (Maas *et al.*, 2013) fixes α_i to a small value like 0.01, while a **parametric ReLU**, or **PReLU**, treats α_i as a learnable parameter (He *et al.*, 2015).

Maxout units (Goodfellow *et al.*, 2013a) generalize rectified linear units further. Instead of applying an element-wise function $g(z)$, maxout units divide $\mathbf{z}$ into groups of k values. Each maxout unit then outputs the maximum element of one of these groups:

$$g(\mathbf{z})_i = \max_{j \in \mathbb{G}^{(i)}} z_j, \quad (6.37)$$

where $\mathbb{G}^{(i)}$ is the set of indices into the inputs for group i , $\{(i-1)k+1, \dots, ik\}$. This provides a way of learning a piecewise linear function that responds to multiple directions in the input $\mathbf{x}$ space.

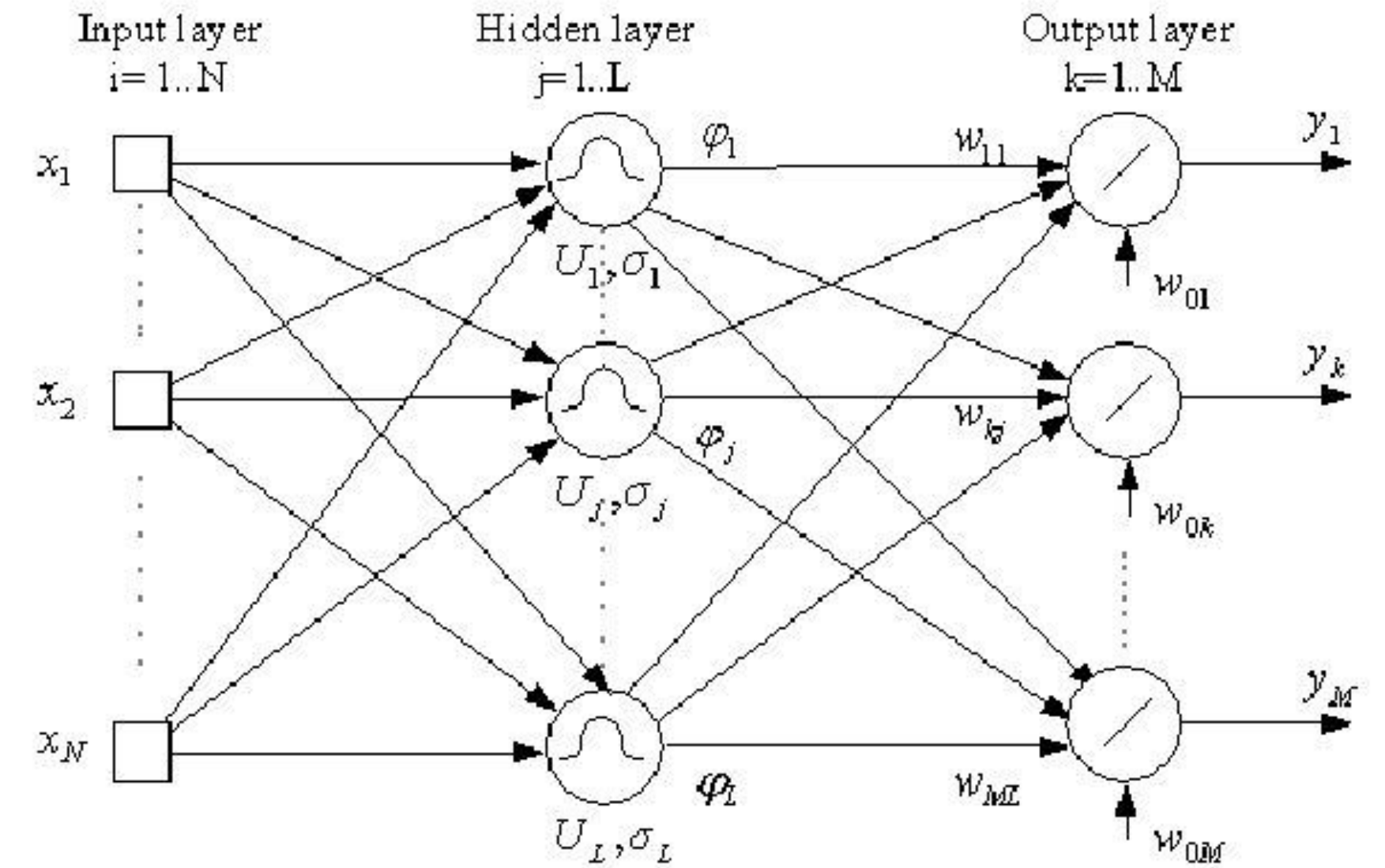
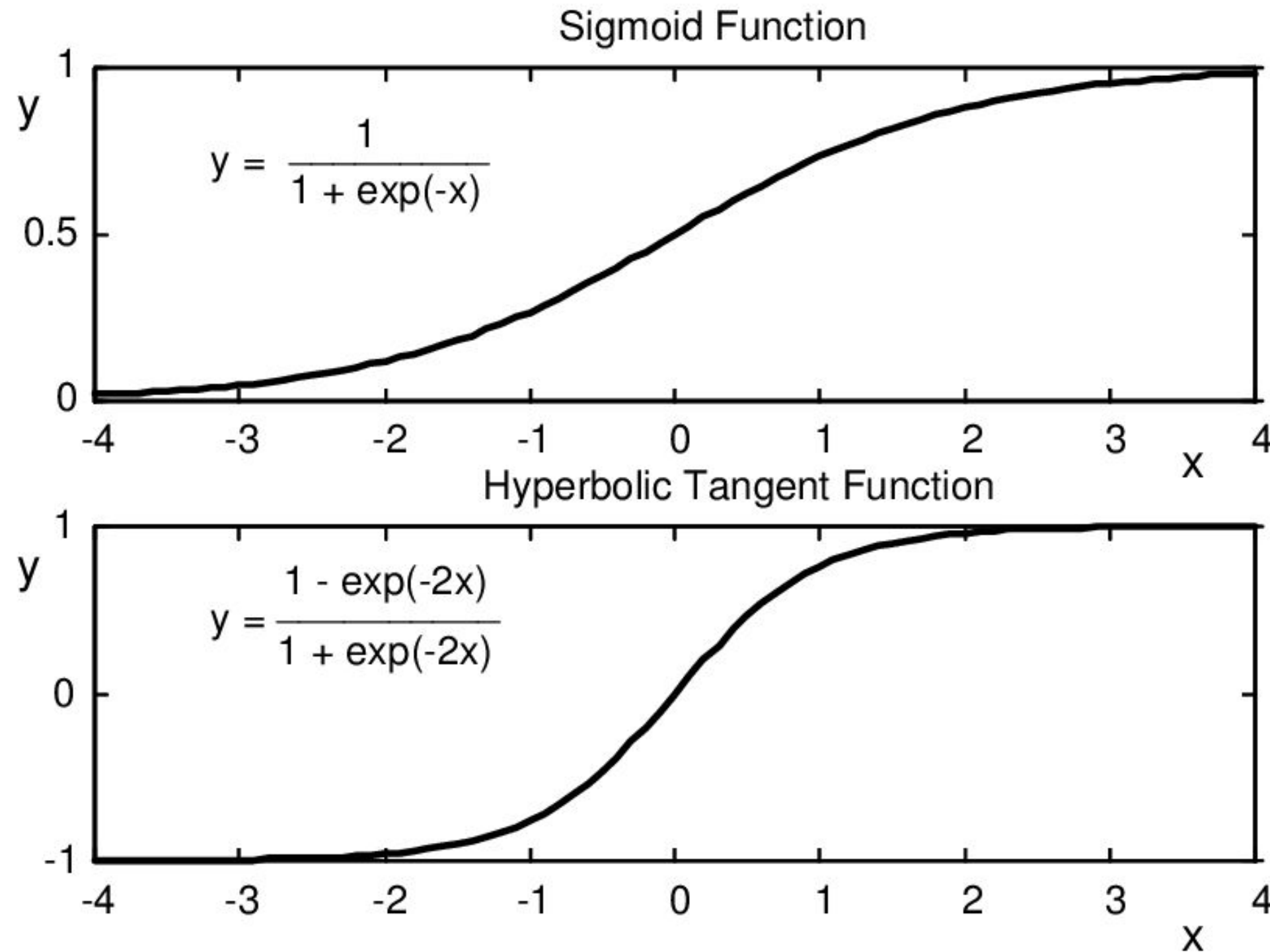
Logistic Sigmoid and Hyperbolic Tangent : Prior to the introduction of rectified linear units, most neural networks used the logistic sigmoid activation function $g(z) = \sigma(z)$ or the hyperbolic tangent activation function $g(z) = \tanh(z)$.

These activation functions are closely related because $\tanh(z) = 2\sigma(2z) - 1$.

Sigmoidal units saturate across most of their domain—they saturate to a high value when z is very positive, saturate to a low value when z is very negative, and are only strongly sensitive to their input when z is near 0.

The widespread saturation of sigmoidal units can make gradient-based learning very difficult.

For this reason, their use as hidden units in feedforward networks is now discouraged



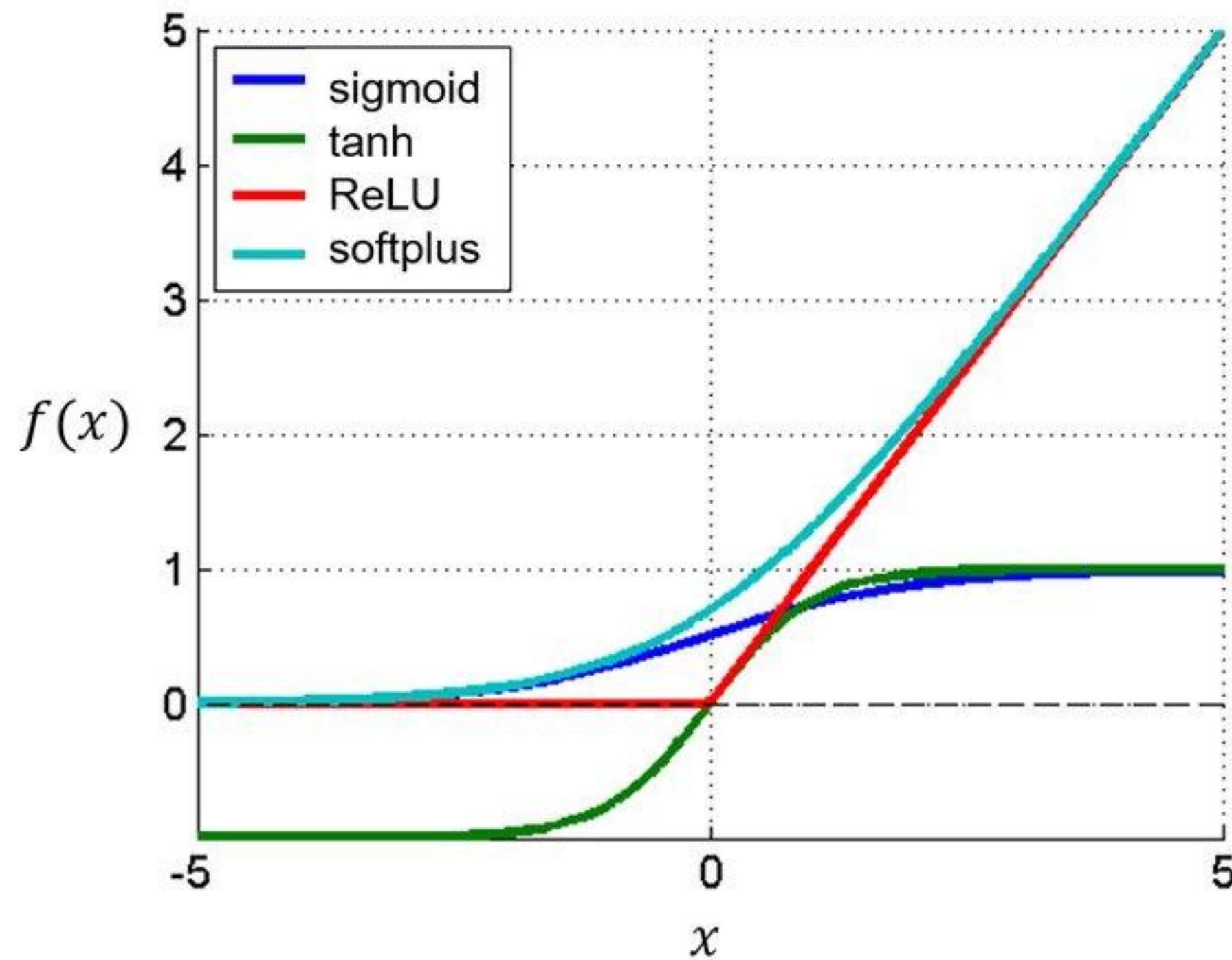
When a sigmoidal activation function must be used, the hyperbolic tangent activation function typically performs better than the logistic sigmoid. It resembles the identity function more closely, in the sense that $\tanh(0) = 0$ while $\sigma(0) = \frac{1}{2}$. Because $\tanh$ is similar to the identity function near 0, training a deep neural network $\hat{y} = \mathbf{w}^\top \tanh(\mathbf{U}^\top \tanh(\mathbf{V}^\top \mathbf{x}))$ resembles training a linear model $\hat{y} =$

$\mathbf{w}^\top \mathbf{U}^\top \mathbf{V}^\top \mathbf{x}$ as long as the activations of the network can be kept small. This makes training the $\tanh$ network easier.

Sigmoidal activation functions are more common in settings other than feed-forward networks. Recurrent networks, many probabilistic models, and some autoencoders have additional requirements that rule out the use of piecewise linear activation functions and make sigmoidal units more appealing despite the drawbacks of saturation.

A few other reasonably common hidden unit types include

- **Radial basis function** (RBF), unit: $h_i = \exp\left(-\frac{1}{\sigma_i^2} \|\mathbf{W}_{:,i} - \mathbf{x}\|^2\right)$. This function becomes more active as $\mathbf{x}$ approaches a template $\mathbf{W}_{:,i}$. Because it saturates to 0 for most $\mathbf{x}$, it can be difficult to optimize.
- **Softplus**: $g(a) = \zeta(a) = \log(1 + e^a)$. This is a smooth version of the rectifier, introduced by [Dugas et al. \(2001\)](#) for function approximation and by [Nair and Hinton \(2010\)](#) for the conditional distributions of undirected probabilistic models. [Glorot et al. \(2011a\)](#) compared the softplus and rectifier and found better results with the latter. The use of the softplus is generally discouraged. The softplus demonstrates that the performance of hidden unit types can be very counterintuitive—one might expect it to have an advantage over the rectifier due to being differentiable everywhere or due to saturating less completely, but empirically it does not.
- **Hard tanh**. This is shaped similarly to the tanh and the rectifier, but unlike the latter, it is bounded, $g(a) = \max(-1, \min(1, a))$. It was introduced by [Collobert \(2004\)](#).



Plot of the sigmoid function, hyperbolic tangent, rectified linear unit layer, and the softplus function. It can be seen that the softplus function is the smooth version of the rectified linear unit layer.

Hardtanh Activation

[Edit](#)

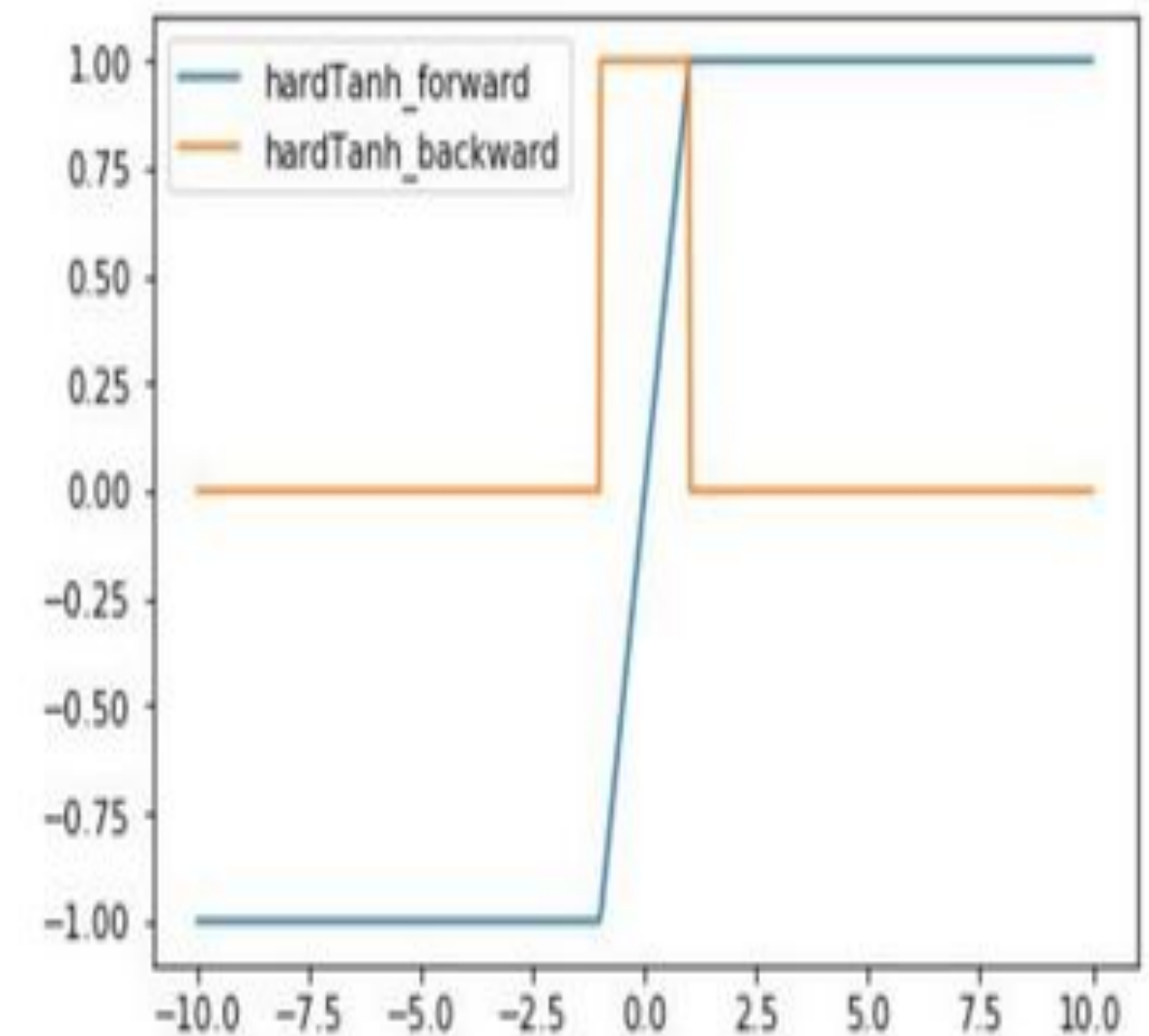
Hardtanh is an activation function used for neural networks:

$$f(x) = -1 \text{ if } x < -1$$

$$f(x) = x \text{ if } -1 \leq x \leq 1$$

$$f(x) = 1 \text{ if } x > 1$$

It is a cheaper and more computationally efficient version of the [tanh activation](#).



6.4 Architecture Design : The word *architecture* refers to the overall structure of the network: how many units it should have and how these units should be connected to each other
Most

neural network architectures arrange these layers in a chain structure, with each layer being a function of the layer that preceded it. In this structure, the first layer is given by

$$h^{(1)} = g^{(1)} \left(W^{(1)\top} x + b^{(1)} \right), \quad (6.40)$$

the second layer is given by

$$h^{(2)} = g^{(2)} \left(W^{(2)\top} h^{(1)} + b^{(2)} \right), \quad (6.41)$$

main architectural considerations are to choose the depth of the network and the width of each layer.

Deeper networks often use fewer units per layer and far fewer parameters and often generalize to the test set, but are also often harder to optimize.

network architecture must be found via experimentation guided by monitoring the validation set error.

the ***universal approximation theorem*** (Hornik et al., 1989; Cybenko, 1989) : states that a feedforward network with a linear output layer and at least one hidden layer with any “squashing” activation function (such as the logistic sigmoid activation function) can approximate any Borel measurable function from one finite-dimensional space to another with any desired non-zero amount of error, provided that the network is given enough hidden units.

The concept of Borel measurability say that any continuous function on a closed and bounded subset of $\mathbb{R}^n$ is *Borel measurable* and therefore may be approximated by a neural network.

The universal approximation theorem means that regardless of what function we are trying to learn, we know that a large MLP will be able to represent this function.

Even if the MLP is able to represent the function, learning can fail for two different reasons.

1. First, the optimization algorithm used for training may not be able to find the value of the parameters that corresponds to the desired function.
2. Second, the training algorithm might choose the wrong function as a result of overfitting.

There is no universal procedure for examining a training set of specific examples and choosing a function that will generalize to points not in the training set.

In summary, a feedforward network with a single layer is sufficient to represent any function, but the layer may be infeasibly large and may fail to learn and generalize correctly.

In many circumstances, using deeper models can reduce the number of units required to represent the desired function and can reduce the amount of generalization error

6.5 Back-Propagation and Other Differentiation Algorithms:

During training, forward propagation can continue onward until it produces a scalar cost $J(\theta)$.

The back-propagation algorithm (Rumelhart et al., 1986a), often simply called backprop, allows the information from the cost to then flow backward through the network in order to compute the gradient.

back-propagation refers only to the method for computing the gradient, while another algorithm, such as stochastic gradient descent, is used to perform learning using this gradient.

Specifically,

we will describe how to compute the gradient

$\nabla_{\mathbf{x}} f(\mathbf{x}, \mathbf{y})$ for an arbitrary function f , where $\mathbf{x}$ is a set of variables whose derivatives are desired, and $\mathbf{y}$ is an additional set of variables that are inputs to the function but whose derivatives are not required. In learning algorithms, the gradient we most often require is the gradient of the cost function with respect to the parameters, $\nabla_{\theta} J(\theta)$. Many machine learning tasks involve computing other derivatives, either as part of the learning process, or to analyze the learned model. The back-propagation algorithm can be applied to these tasks as well and is not restricted to computing the gradient of the cost function with respect to the parameters. The

6.5.1 Computational Graphs : To describe the back-propagation algorithm more precisely, it is helpful to have a more precise computational graph language.

Go, change the world

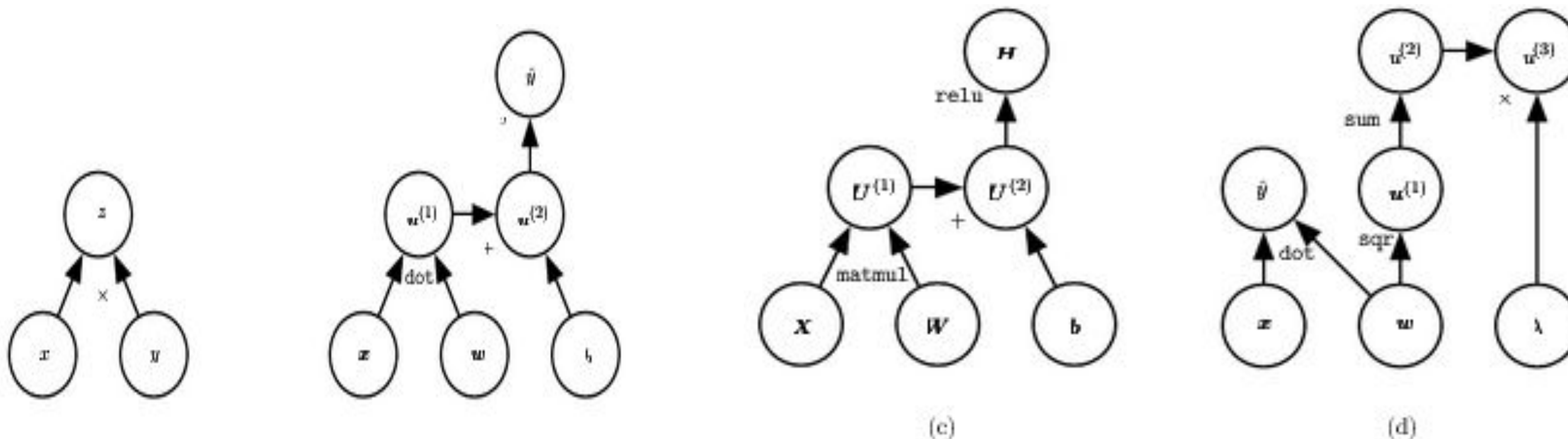


Figure 6.8: Examples of computational graphs. (a) The graph using the $\times$ operation to compute $z = xy$. (b) The graph for the logistic regression prediction $\hat{y} = \sigma(\mathbf{x}^T \mathbf{w} + b)$. Some of the intermediate expressions do not have names in the algebraic expression but need names in the graph. We simply name the i -th such variable $u^{(i)}$. (c) The computational graph for the expression $H = \max\{0, \mathbf{XW} + b\}$, which computes a design matrix of rectified linear unit activations H given a design matrix containing a minibatch of inputs X . (d) Examples a–c applied at most one operation to each variable, but it is possible to apply more than one operation. Here we show a computation graph that applies more than one operation to the weights w of a linear regression model. The weights are used to make both the prediction $\hat{y}$ and the weight decay penalty $\lambda \sum_i w_i^2$.

Let x be a real number, and let f and g both be functions mapping from a real number to a real number. Suppose that $y = g(x)$ and $z = f(g(x)) = f(y)$. Then the chain rule states that

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}. \quad (6.44)$$

We can generalize this beyond the scalar case. Suppose that $\mathbf{x} \in \mathbb{R}^m$, $\mathbf{y} \in \mathbb{R}^n$, g maps from $\mathbb{R}^m$ to $\mathbb{R}^n$, and f maps from $\mathbb{R}^n$ to $\mathbb{R}$. If $\mathbf{y} = g(\mathbf{x})$ and $z = f(\mathbf{y})$, then

$$\frac{\partial z}{\partial x_i} = \sum_j \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}. \quad (6.45)$$

In vector notation, this may be equivalently written as

$$\nabla_{\mathbf{x}} z = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \right)^{\top} \nabla_{\mathbf{y}} z, \quad (6.46)$$

where $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ is the $n \times m$ Jacobian matrix of g .

From this we see that the gradient of a variable $\mathbf{x}$ can be obtained by multiplying a Jacobian matrix $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ by a gradient $\nabla_{\mathbf{y}} z$. The back-propagation algorithm consists of performing such a Jacobian-gradient product for each operation in the graph.

To denote the gradient of a value z with respect to a tensor $\mathbf{X}$, we write $\nabla_{\mathbf{X}} z$, just as if $\mathbf{X}$ were a vector. The indices into $\mathbf{X}$ now have multiple coordinates—for example, a 3-D tensor is indexed by three coordinates. We can abstract this away by using a single variable i to represent the complete tuple of indices. For all possible index tuples i , $(\nabla_{\mathbf{X}} z)_i$ gives $\frac{\partial z}{\partial X_i}$. This is exactly the same as how for all possible integer indices i into a vector, $(\nabla_{\mathbf{x}} z)_i$ gives $\frac{\partial z}{\partial x_i}$. Using this notation, we can write the chain rule as it applies to tensors. If $\mathbf{Y} = g(\mathbf{X})$ and $z = f(\mathbf{Y})$, then

$$\nabla_{\mathbf{X}} z = \sum_j (\nabla_{\mathbf{X}} Y_j) \frac{\partial z}{\partial Y_j}. \quad (6.47)$$

Recursively Applying the Chain Rule to Obtain Backprop

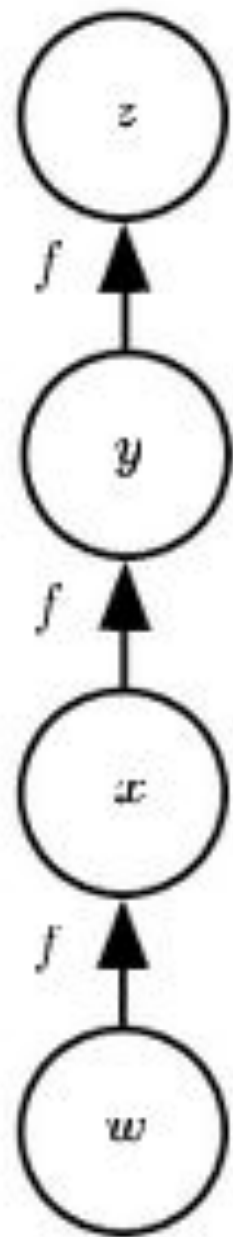


Figure 6.9: A computational graph that results in repeated subexpressions when computing the gradient. Let $w \in \mathbb{R}$ be the input to the graph. We use the same function $f : \mathbb{R} \rightarrow \mathbb{R}$ as the operation that we apply at every step of a chain: $x = f(w)$, $y = f(x)$, $z = f(y)$. To compute $\frac{\partial z}{\partial w}$, we apply equation 6.44 and obtain:

$$\frac{\partial z}{\partial w} \tag{6.49}$$

$$= \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \frac{\partial x}{\partial w} \tag{6.50}$$

$$= f'(y) f'(x) f'(w) \tag{6.51}$$

$$= f'(f(f(w))) f'(f(w)) f'(w). \tag{6.52}$$

Algorithm 6.1 A procedure that performs the computations mapping n_i inputs $u^{(1)}$ to $u^{(n_i)}$ to an output $u^{(n)}$. This defines a computational graph where each node computes numerical value $u^{(i)}$ by applying a function $f^{(i)}$ to the set of arguments $\mathbb{A}^{(i)}$ that comprises the values of previous nodes $u^{(j)}$, $j < i$, with $j \in Pa(u^{(i)})$. The input to the computational graph is the vector $\mathbf{x}$, and is set into the first n_i nodes $u^{(1)}$ to $u^{(n_i)}$. The output of the computational graph is read off the last (output) node $u^{(n)}$.

```
for  $i = 1, \dots, n_i$  do
     $u^{(i)} \leftarrow x_i$ 
end for
for  $i = n_i + 1, \dots, n$  do
     $\mathbb{A}^{(i)} \leftarrow \{u^{(j)} \mid j \in Pa(u^{(i)})\}$ 
     $u^{(i)} \leftarrow f^{(i)}(\mathbb{A}^{(i)})$ 
end for
return  $u^{(n)}$ 
```

Algorithm 6.3 first shows the forward propagation, which maps parameters to the supervised loss $L(\hat{\mathbf{y}}, \mathbf{y})$ associated with a single (input,target) training example $(\mathbf{x}, \mathbf{y})$, with $\hat{\mathbf{y}}$ the output of the neural network when $\mathbf{x}$ is provided in input.

Algorithm 6.3 Forward propagation through a typical deep neural network and the computation of the cost function. The loss $L(\hat{\mathbf{y}}, \mathbf{y})$ depends on the output $\hat{\mathbf{y}}$ and on the target $\mathbf{y}$ (see section 6.2.1.1 for examples of loss functions). To obtain the total cost J , the loss may be added to a regularizer $\Omega(\theta)$, where θ contains all the parameters (weights and biases). Algorithm 6.4 shows how to compute gradients of J with respect to parameters $\mathbf{W}$ and $\mathbf{b}$. For simplicity, this demonstration uses only a single input example $\mathbf{x}$. Practical applications should use a minibatch. See section 6.5.7 for a more realistic demonstration.

Algorithm 6.3 Forward propagation through a typical deep neural network and the computation of the cost function. The loss $L(\hat{\mathbf{y}}, \mathbf{y})$ depends on the output $\hat{\mathbf{y}}$ and on the target $\mathbf{y}$ (see section 6.2.1.1 for examples of loss functions). To obtain the total cost J , the loss may be added to a regularizer $\Omega(\theta)$, where θ contains all the parameters (weights and biases). Algorithm 6.4 shows how to compute gradients of J with respect to parameters $\mathbf{W}$ and $\mathbf{b}$. For simplicity, this demonstration uses only a single input example $\mathbf{x}$. Practical applications should use a minibatch. See section 6.5.7 for a more realistic demonstration.

Require: Network depth, l

Require: $\mathbf{W}^{(i)}, i \in \{1, \dots, l\}$, the weight matrices of the model

Require: $\mathbf{b}^{(i)}, i \in \{1, \dots, l\}$, the bias parameters of the model

Require: $\mathbf{x}$, the input to process

Require: $\mathbf{y}$, the target output

$$\mathbf{h}^{(0)} = \mathbf{x}$$

for $k = 1, \dots, l$ **do**

$$\mathbf{a}^{(k)} = \mathbf{b}^{(k)} + \mathbf{W}^{(k)} \mathbf{h}^{(k-1)}$$

$$\mathbf{h}^{(k)} = f(\mathbf{a}^{(k)})$$

end for

$$\hat{\mathbf{y}} = \mathbf{h}^{(l)}$$

$$J = L(\hat{\mathbf{y}}, \mathbf{y}) + \lambda \Omega(\theta)$$

Algorithm 6.4 Backward computation for the deep neural network of algorithm 6.3, which uses, in addition to the input $\mathbf{x}$, a target $\mathbf{y}$. This computation yields the gradients on the activations $\mathbf{a}^{(k)}$ for each layer k , starting from the output layer and going backwards to the first hidden layer. From these gradients, which can be interpreted as an indication of how each layer's output should change to reduce error, one can obtain the gradient on the parameters of each layer. The gradients on weights and biases can be immediately used as part of a stochastic gradient update (performing the update right after the gradients have been computed) or used with other gradient-based optimization methods.

After the forward computation, compute the gradient on the output layer:

$$\mathbf{g} \leftarrow \nabla_{\mathbf{y}} J = \nabla_{\mathbf{y}} L(\hat{\mathbf{y}}, \mathbf{y})$$

for $k = l, l - 1, \dots, 1$ **do**

Convert the gradient on the layer's output into a gradient on the pre-nonlinearity activation (element-wise multiplication if f is element-wise):

$$\mathbf{g} \leftarrow \nabla_{\mathbf{a}^{(k)}} J = \mathbf{g} \odot f'(\mathbf{a}^{(k)})$$

Compute gradients on weights and biases (including the regularization term, where needed):

$$\nabla_{\mathbf{b}^{(k)}} J = \mathbf{g} + \lambda \nabla_{\mathbf{b}^{(k)}} \Omega(\theta)$$

$$\nabla_{\mathbf{W}^{(k)}} J = \mathbf{g} \mathbf{h}^{(k-1)\top} + \lambda \nabla_{\mathbf{W}^{(k)}} \Omega(\theta)$$

Propagate the gradients w.r.t. the next lower-level hidden layer's activations:

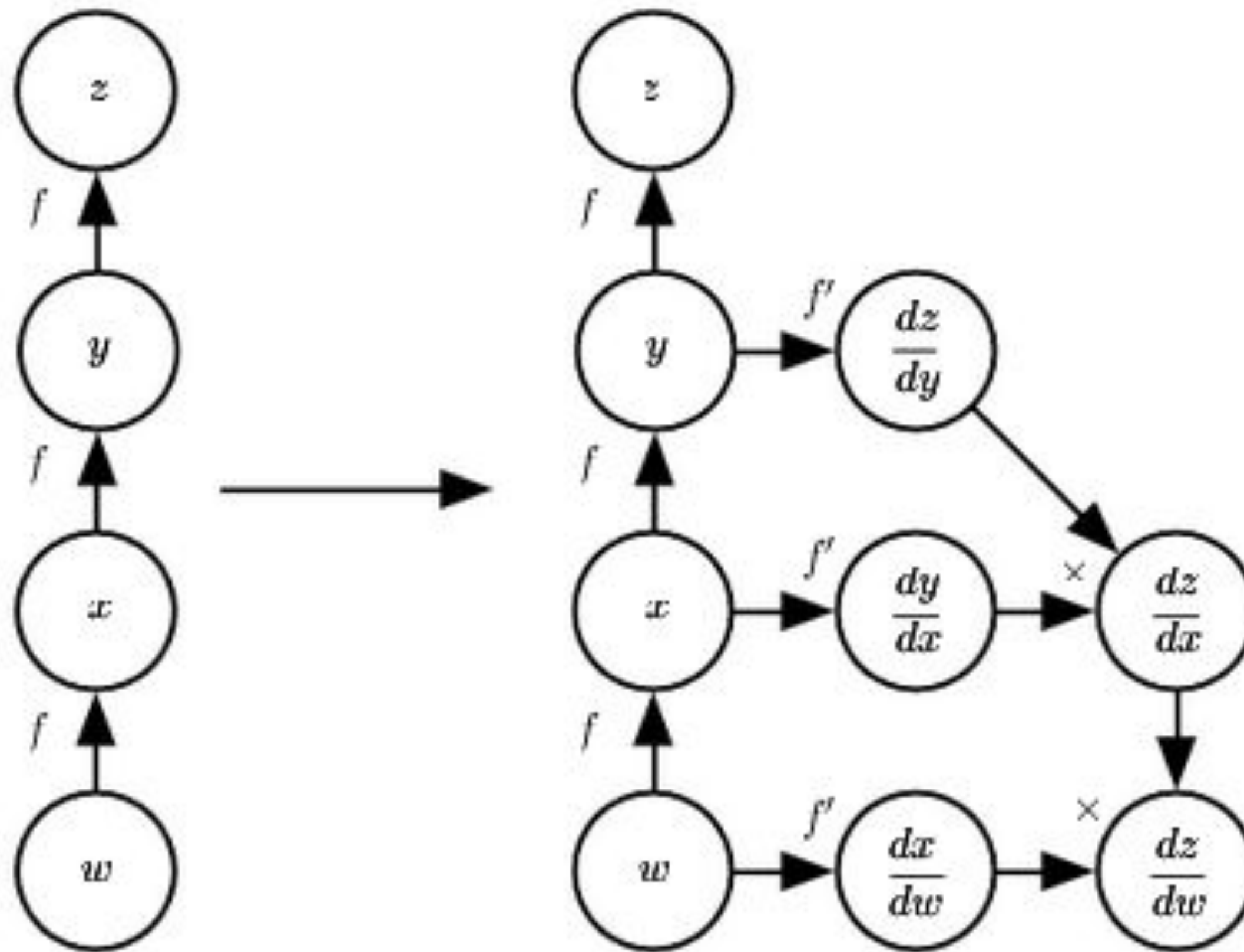
$$\mathbf{g} \leftarrow \nabla_{\mathbf{h}^{(k-1)}} J = \mathbf{W}^{(k)\top} \mathbf{g}$$

end for

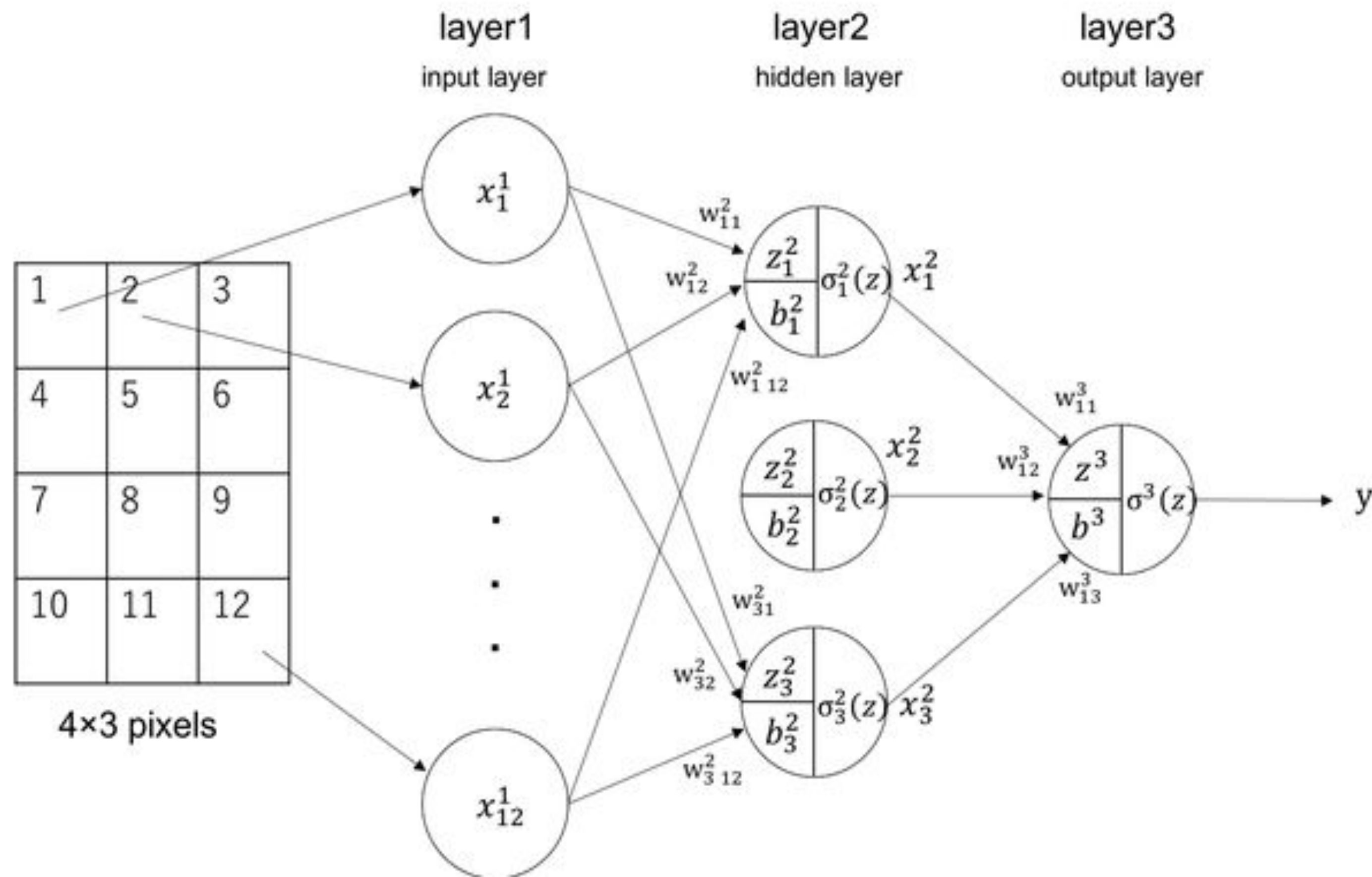
These algebraic and graph-based representations are called **symbolic representations** . When we actually use or train a neural network, we must assign specific values to these symbols. We replace a symbolic input to the network x with a specific numeric value, such as **[1.2, 3.765, -1.8]**.

Some approaches to back-propagation take a computational graph and a set of numerical values for the inputs to the graph, then return a set of numerical values describing the gradient at those input values. We call this approach symbol-to- number differentiation

representing $z = f(f(f(w)))$. (Right) We run the back-propagation algorithm, instructing it to construct the graph for the expression corresponding to $\frac{dz}{dw}$. In this example, we do not explain how the back-propagation algorithm works. The purpose is only to illustrate what the desired result is: a computational graph with a symbolic description of the derivative.

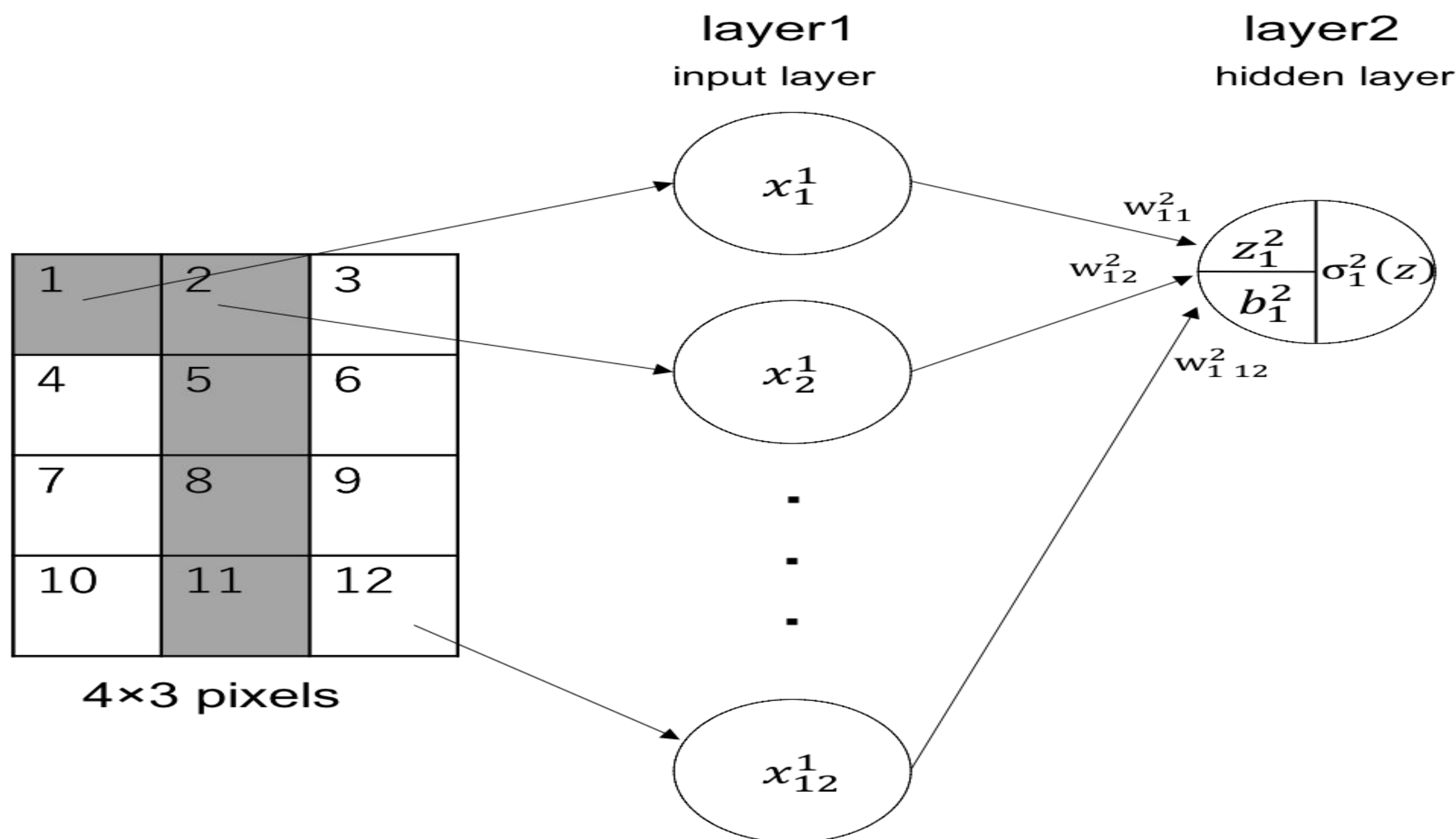


$$x_i^1 = \begin{cases} 0 \\ 1 \end{cases}$$



4x3 pixels

$$\begin{aligned} x_1^1 &= 1, & x_2^1 &= 1, & x_3^1 &= 0 \\ x_4^1 &= 0, & x_5^1 &= 1, & x_6^1 &= 0 \\ x_7^1 &= 0, & x_8^1 &= 1, & x_9^1 &= 0 \\ x_{10}^1 &= 0, & x_{11}^1 &= 1, & x_{12}^1 &= 0 \end{aligned}$$



$$w_{11}^2 = 0.48, w_{12}^2 = 0.29, w_{13}^2 = 0.58, w_{14}^2 = 0.61, w_{15}^2 = 0.47, w_{16}^2 = 0.18,$$

$$w_{17}^2 = 0.02, w_{18}^2 = 0.49, w_{19}^2 = 0.67, w_{1\ 10}^2 = 0.62, w_{1\ 11}^2 = 0.017, w_{1\ 12}^2 = 0.28$$

$$b_1^2 = 0$$

The weighted input would be as follows:

$$\begin{aligned} z_1^2 &= x_1^1 w_{11}^2 + x_2^1 w_{12}^2 + x_3^1 w_{13}^2 + x_4^1 w_{14}^2 + x_5^1 w_{15}^2 + x_6^1 w_{16}^2 + x_7^1 w_{17}^2 \\ &\quad + x_8^1 w_{18}^2 + x_9^1 w_{19}^2 + x_{10}^1 w_{1\ 10}^2 + x_{11}^1 w_{1\ 11}^2 + x_{12}^1 w_{1\ 12}^2 + b_1^2 \\ &= 1 \times w_{11}^2 + 1 \times w_{12}^2 + 0 \times w_{13}^2 + 0 \times w_{14}^2 + 1 \times w_{15}^2 + 0 \times w_{16}^2 + 0 \times w_{17}^2 \\ &\quad + 1 \times w_{18}^2 + 0 \times w_{19}^2 + 0 \times w_{1\ 10}^2 + 1 \times w_{1\ 11}^2 + 0 \times w_{1\ 12}^2 + b_1^2 \\ &= 1 \times 0.48 + 1 \times 0.29 + 0 \times 0.58 + 0 \times 0.61 + 1 \times 0.47 + 0 \times 0.18 + 0 \times \\ &\quad + 1 \times 0.49 + 0 \times 0.67 + 0 \times 0.62 + 1 \times 0.017 + 0 \times 0.28 + 0 \\ &= 1 \times 0.48 + 1 \times 0.29 + 1 \times 0.47 + 1 \times 0.49 + 1 \times 0.017 + 0 \\ &= 1.277 \end{aligned}$$

We insert the value into an activation function. We will use sigmoid function for the same reason in section 1. The output of the unit will be,

$$\begin{aligned}x_1^2 &= \sigma_1^2(z_1^2) = \frac{1}{1+\exp(-z_1^2)} \\&= \frac{1}{1+\exp(-(1.277))} \\&= 0.782\end{aligned}$$

The same computations are conducted in other units of the neural network.

Here, we will introduce variables and parameters which are commonly used in thesis or interpretations. The figure would help you grasp more deeply.

x_i^{l-1} : The output from unit i in layer l-1. It becomes the inputs of the units in layer l.

x_j^l : The output from unit j in layer l. It becomes the inputs of the units in layer l+1.

w_{ji}^l : The weight bears on the signal which is outputted from unit i in layer l-1 and goes in to unit j in layer l.

b_j^l : The bias bears on unit j in layer l.

z_j^l : The weighted input of unit j in layer l.

$\sigma_j^l(z_j^l)$: The activation function of unit j in layer l

The following are examples:

x_2^1 : The output from unit 2 in layer 1. It becomes the inputs of the units in layer 2.

x_3^2 : The output from unit 3 in layer 2. It becomes the inputs of the units in layer 3.

w_{31}^2 : The weight bears on the signal which is outputted from the unit No. 1 in layer 1 and goes in to unit 3 in layer 2.

b_3^2 : The bias bears on unit 3 in layer 2.

z_3^2 : The weighted input of unit 3 in layer 2.

$\sigma_3^2(z_3^2)$: The activation function of unit 3 in layer 2

we initialize the weights with random values (0~1) and the bias as 0. Unit 1 of Layer 2 has the same weight as the previous.

$$\begin{aligned}w_{11}^2 &= 0.48, w_{12}^2 = 0.29, w_{13}^2 = 0.58, w_{14}^2 = 0.61, w_{15}^2 = 0.47, w_{16}^2 = 0.18, \\w_{17}^2 &= 0.02, w_{18}^2 = 0.49, w_{19}^2 = 0.67, w_{1\ 10}^2 = 0.62, w_{1\ 11}^2 = 0.017, w_{1\ 12}^2 = 0.28 \\w_{21}^2 &= 0.52, w_{22}^2 = 0.036, w_{23}^2 = 0.90, w_{24}^2 = 0.099, w_{25}^2 = 0.46, w_{26}^2 = 0.87, \\w_{27}^2 &= 0.99, w_{28}^2 = 0.83, w_{29}^2 = 0.15, w_{2\ 10}^2 = 0.14, w_{2\ 11}^2 = 0.64, w_{2\ 12}^2 = 0.88 \\w_{31}^2 &= 0.48, w_{32}^2 = 0.055, w_{33}^2 = 0.56, w_{34}^2 = 0.16, w_{35}^2 = 0.86, w_{36}^2 = 0.34, \\w_{37}^2 &= 0.40, w_{38}^2 = 0.06, w_{39}^2 = 0.66, w_{3\ 10}^2 = 0.72, w_{3\ 11}^2 = 0.077, w_{3\ 12}^2 = 0.29 \\b_1^2 &= 0 \\b_2^2 &= 0 \\b_3^2 &= 0\end{aligned}$$

The equations below are the 3 weighted inputs of the hidden layer.

$$\left\{ \begin{array}{l} z_1^2 = x_1^1 w_{11}^2 + x_2^1 w_{12}^2 + x_3^1 w_{13}^2 + x_4^1 w_{14}^2 + x_5^1 w_{15}^2 + x_6^1 w_{16}^2 + x_7^1 w_{17}^2 \\ \quad + x_8^1 w_{18}^2 + x_9^1 w_{19}^2 + x_{10}^1 w_{1\ 10}^2 + x_{11}^1 w_{1\ 11}^2 + x_{12}^1 w_{1\ 12}^2 + b_1^2 \\ z_2^2 = x_1^1 w_{21}^2 + x_2^1 w_{22}^2 + x_3^1 w_{23}^2 + x_4^1 w_{24}^2 + x_5^1 w_{25}^2 + x_6^1 w_{26}^2 + x_7^1 w_{27}^2 \\ \quad + x_8^1 w_{28}^2 + x_9^1 w_{29}^2 + x_{10}^1 w_{2\ 10}^2 + x_{11}^1 w_{2\ 11}^2 + x_{12}^1 w_{2\ 12}^2 + b_2^2 \\ z_3^2 = x_1^1 w_{31}^2 + x_2^1 w_{32}^2 + x_3^1 w_{33}^2 + x_4^1 w_{34}^2 + x_5^1 w_{35}^2 + x_6^1 w_{36}^2 + x_7^1 w_{37}^2 \\ \quad + x_8^1 w_{38}^2 + x_9^1 w_{39}^2 + x_{10}^1 w_{3\ 10}^2 + x_{11}^1 w_{3\ 11}^2 + x_{12}^1 w_{3\ 12}^2 + b_3^2 \end{array} \right.$$

We can perceive each z consisting the same inputs x , but different weights w and bias b . Let's insert the values to the variables.

$$\Leftrightarrow \begin{cases} z_1^2 = 1 \times 0.48 + 1 \times 0.29 + 0 \times 0.58 + 0 \times 0.61 + 1 \times 0.47 + 0 \times 0.18 + 0 \times 0.02 \\ \quad + 1 \times 0.49 + 0 \times 0.67 + 0 \times 0.62 + 1 \times 0.017 + 0 \times 0.28 + 0 \\ z_2^2 = 1 \times 0.52 + 1 \times 0.039 + 0 \times 0.90 + 0 \times 0.099 + 1 \times 0.46 + 0 \times 0.87 + 0 \times 0.99 \\ \quad + 1 \times 0.83 + 0 \times 0.15 + 0 \times 0.14 + 1 \times 0.64 + 0 \times 0.88 + 0 \\ z_3^2 = 1 \times 0.48 + 1 \times 0.055 + 0 \times 0.56 + 0 \times 0.16 + 1 \times 0.86 + 0 \times 0.34 + 0 \times 0.40 \end{cases}$$

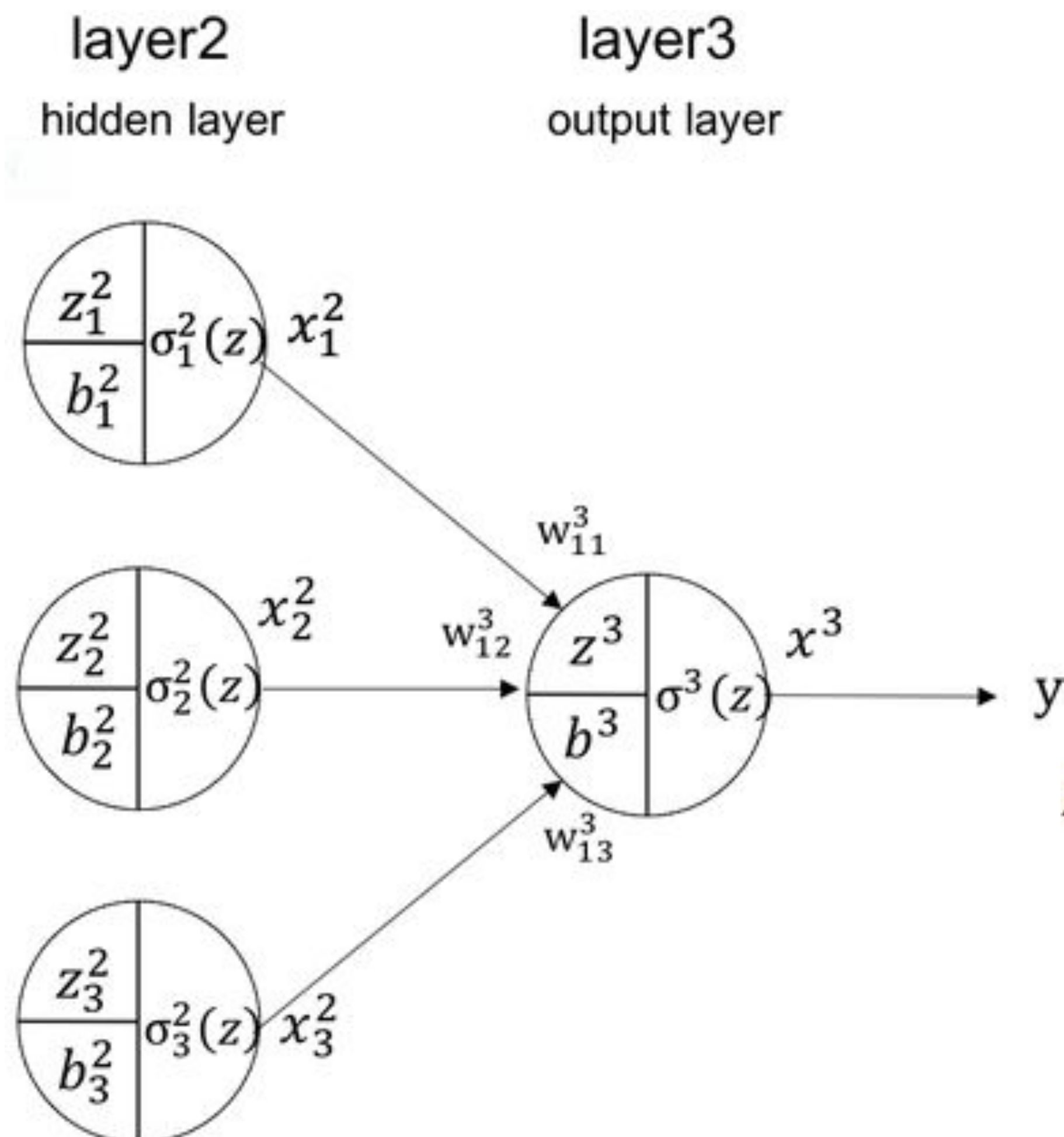
$$\Leftrightarrow \begin{cases} z_1^2 = 1 \times 0.48 + 1 \times 0.29 + 1 \times 0.47 + 1 \times 0.49 + 1 \times 0.017 + 0 \\ z_2^2 = 1 \times 0.52 + 1 \times 0.039 + 1 \times 0.46 + 1 \times 0.83 + 1 \times 0.64 + 0 \\ z_3^2 = 1 \times 0.48 + 1 \times 0.055 + 1 \times 0.86 + 1 \times 0.06 + 1 \times 0.077 + 0 \end{cases}$$

$$\Leftrightarrow \begin{cases} z_1^2 = 1.277 \\ z_2^2 = 2.489 \\ z_3^2 = 1.532 \end{cases}$$

The results shown above are inputs to the activation functions. As mentioned before, we are using sigmoid function. The outputs of the 3 units will become as follows:

$$\begin{cases} x_1^2 = \sigma_1^2(z_1^2) = \frac{1}{1+\exp(-z_1^2)} = \frac{1}{1+\exp(-(1.277))} = \mathbf{0.782} \\ x_2^2 = \sigma_2^2(z_2^2) = \frac{1}{1+\exp(-z_2^2)} = \frac{1}{1+\exp(-(2.489))} = \mathbf{0.923} \\ x_3^2 = \sigma_3^2(z_3^2) = \frac{1}{1+\exp(-z_3^2)} = \frac{1}{1+\exp(-(1.532))} = \mathbf{0.822} \end{cases}$$

Hidden layer → Output layer



the input for the unit will be as follows:

$$w_1^3 = 0.33, w_2^3 = 0.21, w_3^3 = 0.96$$

$$b^3 = 0$$

$$\begin{cases} x_1^2 = 0.782 \\ x_2^2 = 0.923 \\ x_3^2 = 0.822 \end{cases}$$

The weighted input in the output layer will be

$$\begin{aligned} z^3 &= x_1^2 w_{11}^3 + x_2^2 w_{12}^3 + x_3^2 w_{13}^3 + b^3 \\ &= 0.782 \times w_{11}^3 + 0.923 \times w_{12}^3 + 0.822 \times w_{13}^3 + b^3 \\ &= 0.690 \times 0.33 + 0.989 \times 0.21 + 0.550 \times 0.96 + 0 \\ &= 0.963 \end{aligned}$$

The output of the unit will become as follows:

$$x^3 = \sigma^3(z^3) = \frac{1}{1+\exp(-z^3)} = \frac{1}{1+\exp(-(0.963))} = 0.724$$

Sigmoid function has the following properties:

$$1. 0 < \sigma(z) < 1 \quad (\lim_{z \rightarrow -\infty} \sigma(z) = 0, \quad \lim_{z \rightarrow \infty} \sigma(z) = 1)$$

$$2. \sigma(0) = 0.5$$

for classification into 2 states, that is, whether “1” was written on the picture or not. In this situation, we can use the following rule for decision as shown below:

When $y = x^3 = \sigma^3(z^3)$

$$\begin{cases} y \text{ is close to } 1 & (y \geq 0.5) \rightarrow \text{The number on the picture is } 1 \\ y \text{ is close to } 0 & (y < 0.5) \rightarrow \text{The number on the picture is not } 1. \end{cases} \quad y = x^3 = \sigma^3(z^3) = 0.724$$

It is $y \geq 0.5$, thus *y is close to 1*. This means *The number on the picture is 1*.

The input of the picture was 1, so the classification is accurate for this data. Nonetheless, we must note that the parameters (the weights w , the bias b) we assume earlier were not given intentionally, but initialized randomly. In the other words, the parameters were aimlessly set up in the first place and happen to give an accurate result.

When learning with neural network, the neural network adjust its own parameters to give a better accurate classification result. To do this, an algorithm called "Backpropagation" is used to calculate how much update is needed for each weight. In the following tutorial, we will introduce "Backpropagation", using numerical values.

we will show how to calculate backpropagation after values are input. “Backpropagation” is a method of calculating how to modify the parameters to values which make the prediction or the classification of the neural network more accurate. We will consider the same neural network in the previous tutorial of “Calculation example of Feed Forward Neural Network” which is shown below. We will use SGD(Stochastic Gradient Descent) as the optimizer.

Thank You

